

SPM: Shared Memory Systems

Academic year 2025-2026

Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- Understanding the von Neumann bottleneck
- Cache organization and cache algorithms
- Superscalar processors and HW Multithreading
- Introduction to the shared-memory programming model
 - The [data race](#) problem
 - The [false sharing](#) problem
- Characterizing the performance on shared-memory architecture: the [Roofline model](#)

Shared Memory architectures

- Basic structure of the classical von Neumann Architecture
- In early computer systems timings for accessing main memory and for computation were reasonably well balanced
- During the past few decades, computation speed grew much faster than main memory access speed, resulting in a significant performance gap.
- **von Neumann Bottleneck:**
The growing gap between CPU compute speed and memory access speed, where memory bandwidth cannot keep pace with the processor demand for data
 - Things get worse as core count increases (**post-Dennard scaling**), since aggregate bandwidth demand grows while DRAM bandwidth scales more slowly

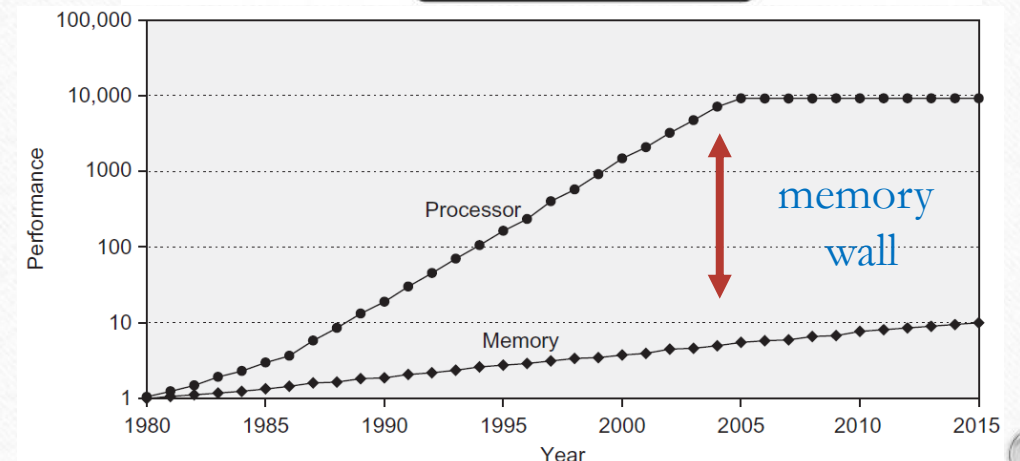
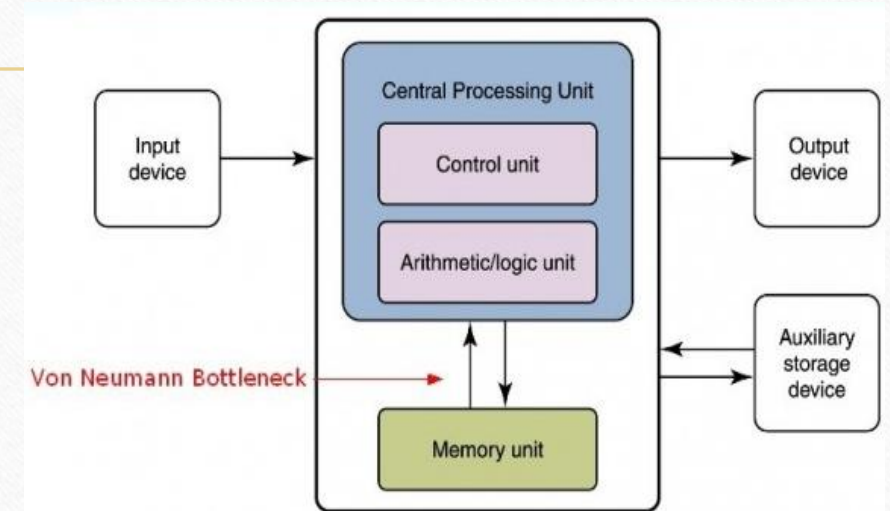


Figure 2.2 from J.L.Hennessy, D.A. Patterson «Computer Architecture A Quantitative Approach» sixth edition

Simplified Architecture

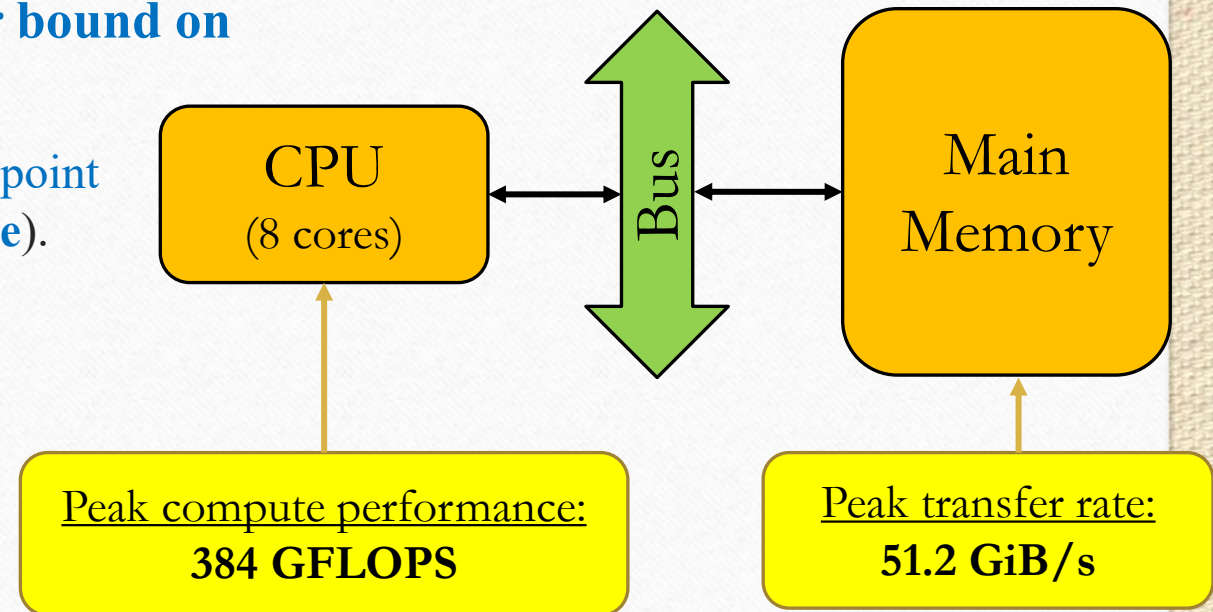
- Simplified architecture model to establish an **upper bound on performance**. Consider a system equipped with:

- 1 CPU, with 8 cores @3GHz capable of 16 **floating point operations per core and per clock cycle (FLOP/cycle)**.

The peak performance is:

$$R_{peak} = 3 \cdot 8 \cdot 16 = 384 \text{ GFLOPS} (*)$$

- The DRAM peak memory transfer rate is 51.2



R_{peak} provides a theoretical peak performance metric for a system assuming perfect conditions (no memory stalls, perfect ILP).

(*) GFLOPS means Giga FLOP per second (GFLOP/s)

Real-world applications often achieve only a small fraction (10-40%) of R_{peak} due to memory bottlenecks, cache misses, and instruction dependencies

von Neumann Bottleneck – Example

- Let's execute the **dot product** kernel on the example platform
- We compute 1 product and 1 sum for each element, i.e., 2 FLOP; we read n elements from $u[]$ and n from $v[]$, thus $2 \cdot n \cdot 8B$ data transferred from memory.
- If $n = 2^{30} \rightarrow 2 \cdot n \text{ FLOP} = 2 \text{ GFLOP}$, and $2 \cdot n \cdot 8B = 16GiB$ data transferred

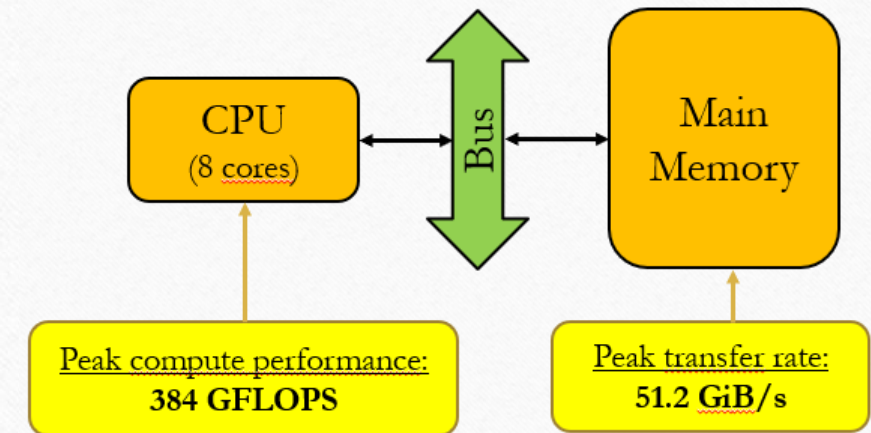
$$t_{comp} = \frac{2 \text{ GFLOP}}{384 \text{ GFLOPS}} = 5.2ms, \quad t_{mem} = \frac{16 \text{ GB}}{51.2 \text{ GiB/s}} = 312.5ms$$

- If we **overlap computation and memory transfer**, a lower bound of the execution time is

$$t_{exec} \geq \max(t_{comp}, t_{mem}) = 312.5ms$$

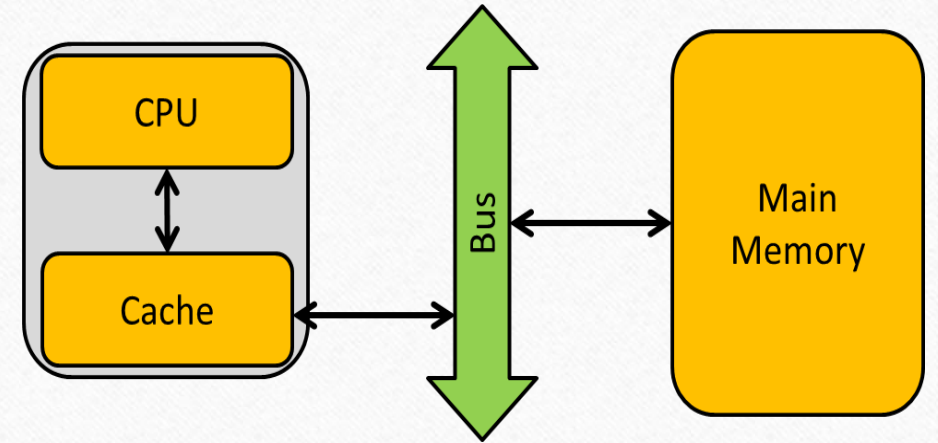
- The achievable performance rate is: $\frac{2 \text{ GFLOP}}{312.5 \text{ ms}} = 6.4 \text{ GFLOPS}$
 - This is less than 2% of peak compute performance
- The dot product kernel is **memory bound** on the example platform, i.e., it is limited by the data transfer rate from main memory

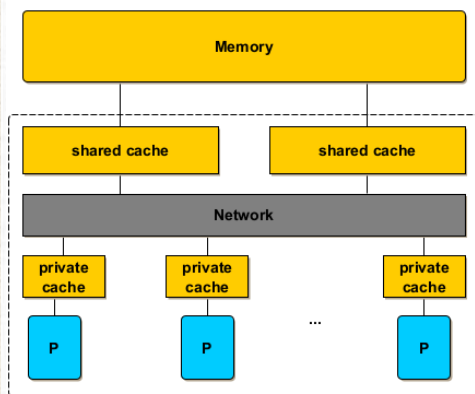
```
// Dot Product (scalar product)
double dotp = 0.0;
for (int i = 0; i < n; i++)
    dotp += u[i] * v[i];
```



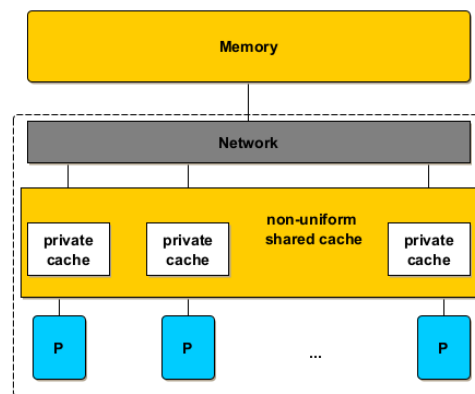
Using caches to mitigate the von Neumann bottleneck

- Modern CPUs typically contain a hierarchy of two or three levels of cache (L1, L2, L3)
- The motivation of cache-based architectures is to add a fast on-chip memory to exploit temporal and spatial locality and mitigate the von Neumann bottleneck
- Caches have higher bandwidth and lower latency compared to main memory but much smaller capacity
- Trade-off between capacity and speed
 - E.g., the L1 cache is small but fast (0.5-1 ns), while the L3 cache is relatively large but much slower than L1 (15-40 ns)
 - Main memory access time is higher (50-100ns)
- Caches can be private for a single core or shared between several cores



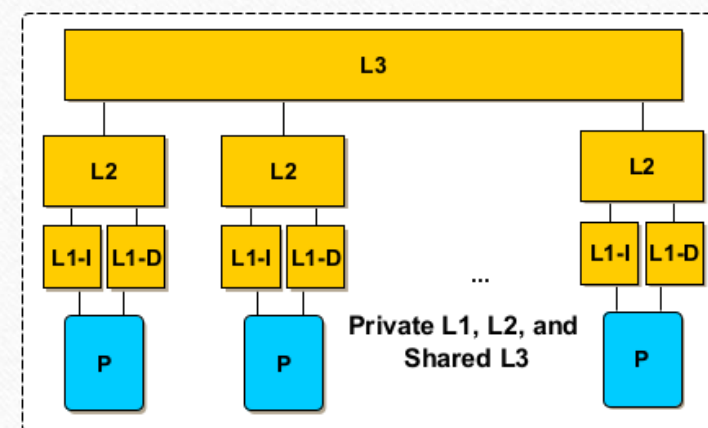


Multi-level cache architecture



Distributed shared-cache architecture

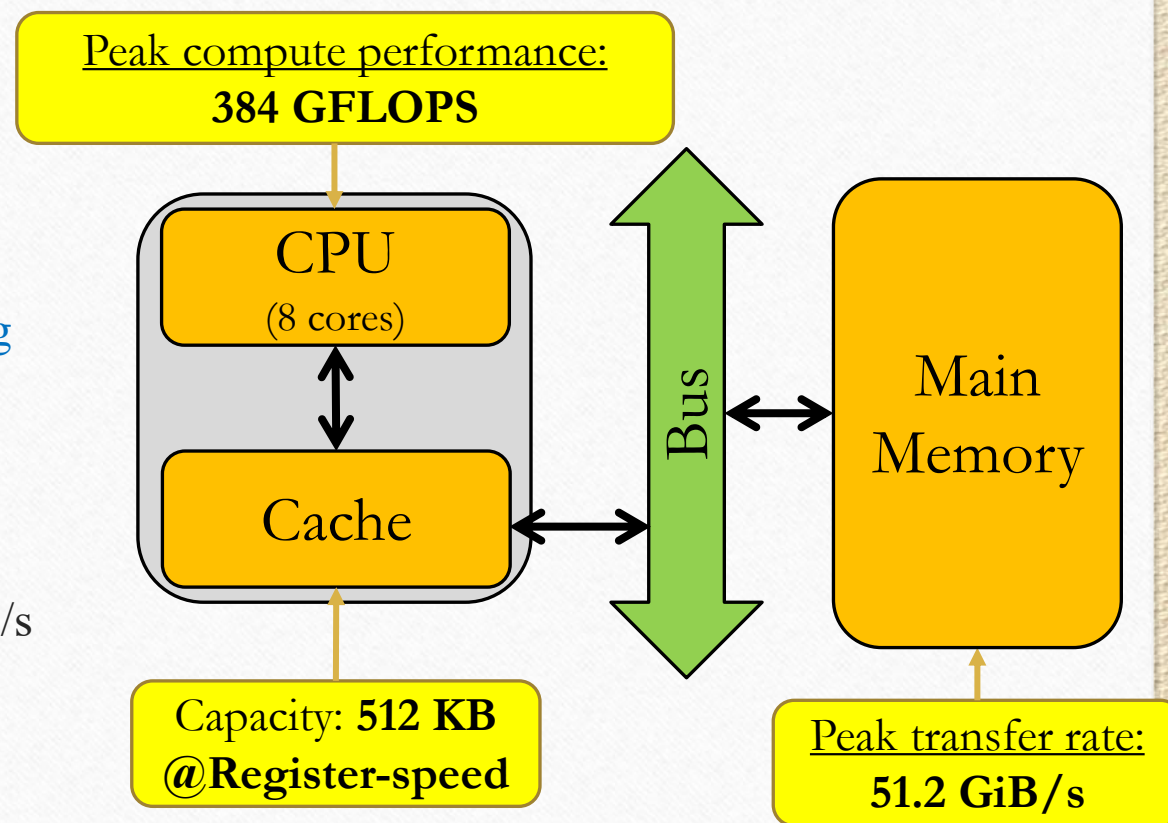
Commonly used cache organization in today's CMPs



CMP	#core	#thr.	GHz	L1	L2	L3
Intel® Xeon Platinum 8380 «Ice Lake»	40	80	2.3	40x 48 KB	40x 1.25 MB	60MB
Amper Altra Ma ARM v8	128	-	3.0	128x 64KB	128x 1 MB	16 MB
AMD® EPYC™ 9654P «Zen 4»	96	192	2.4	6 MB	96x 1 MB	384MB

Cache memory – Example Architecture

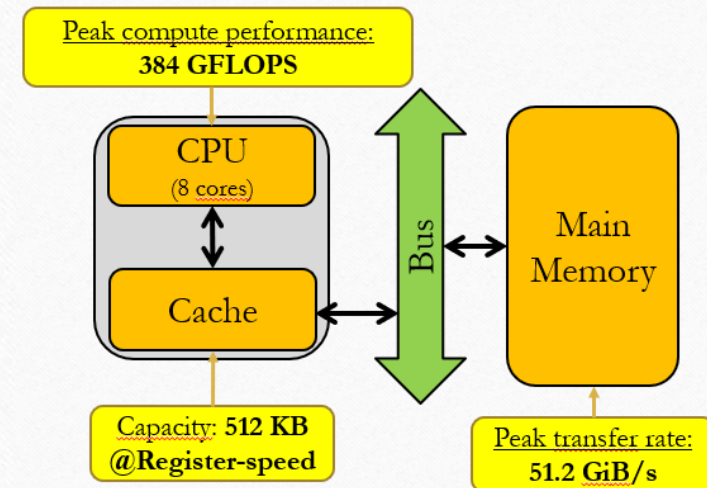
- Simplified model to establish an **upper bound on performance**
- Let's suppose:
 - 1 CPU, with 8 cores @3GHz capable of 16 floating point operations per core per clock cycle (FLOP).
The peak performance is:
$$R_{peak} = 3 \cdot 8 \cdot 16 = 384 \text{ GFLOPS}$$
 - The DRAM peak memory transfer rate is 51.2 GiB/s
 - **On-chip fast shared cache of capacity 512 KB**



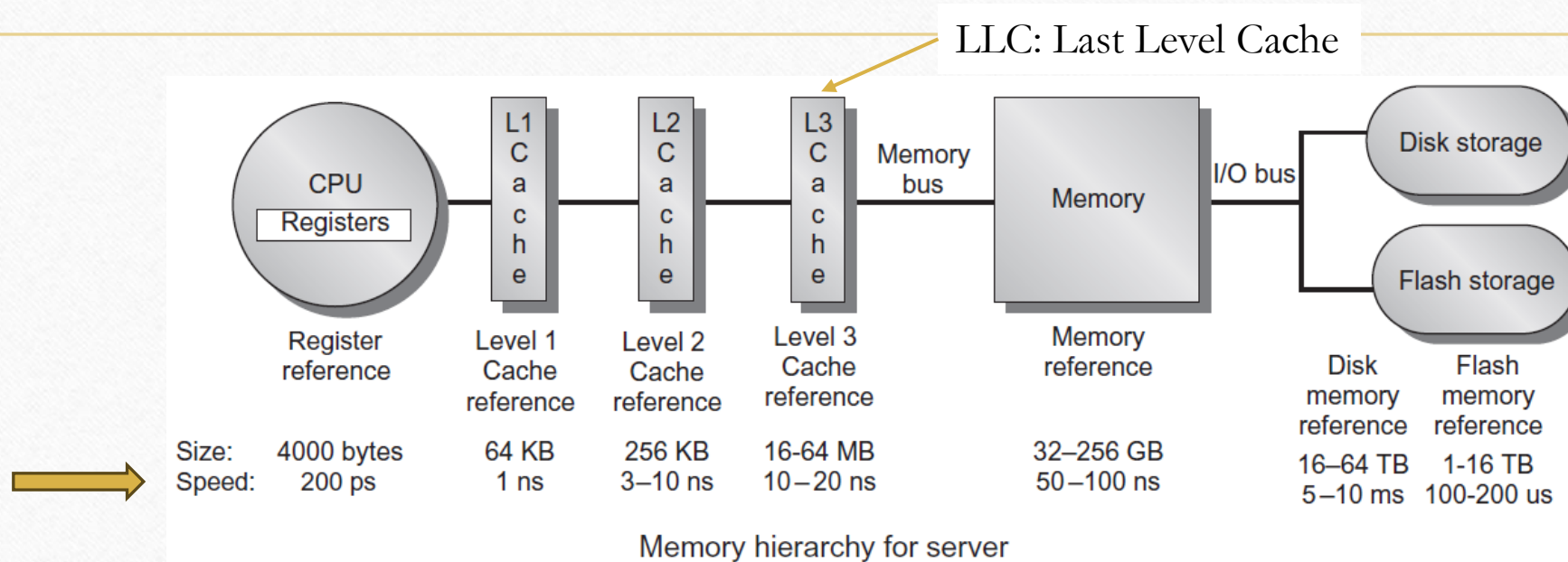
Matrix Multiplication on the Example Arch.

- $W = U \times V$ square matrices with $n = 128$
- Total size of the matrices: $128^2 \cdot 3 \cdot 8B = 384 \text{ KiB}$ (fits in Cache)
- Let's assume we transfer the input matrices U and V from memory to cache once at the beginning, and the matrix W from cache to memory once at the end. This assumes all three matrices fit simultaneously in cache and remain resident throughout the computation.
- Data transfer time (from/to cache): $t_{\text{mem}} = \frac{384 \text{ KB}}{51.2 \text{ GiB/s}} = 7.5 \mu\text{s}$
- Computation time: $t_{\text{comp}} = \frac{2^{22} \text{ FLOP}}{384 \text{ GFLOPS}} = 10.4 \mu\text{s}$
 - Total operations: $2 \cdot n \cdot n^2 = 2 \cdot 128^3 = 2^{22} \text{ FLOP}$
- In this case $t_{\text{mem}} < t_{\text{comp}}$ thus MM is **compute bound** (for $n=128$)
- Execution time: $t_{\text{exec}} \geq 7.5 \mu\text{s} + 10.4 \mu\text{s} = 17.9 \mu\text{s}$
 - Achievable performance (upper bound): $\frac{2^{22} \text{ FLOP}}{17.9 \mu\text{s}} = 223 \text{ GFLOPS}$ ($\approx 60\%$ of peak)
- **What if matrices are bigger than cache?**
We would need to reload data multiple times, increasing t_{mem} . This motivates cache-blocking/tiling techniques (see later slides)

```
//Matrix Multiplication (nxn)
for (int i = 0; i<n; i++)
  for (int j = 0; j < n; j++) {
    double dotp = 0;
    for (int k = 0; k<n; k++)
      dotp += U[i][k]*V[k][j];
    W[i][j] = dotp;
  }
```



Memory Hierarchy



- Exploiting **data locality** makes small fast caches behave like large fast memory
- Key idea: **maximize the reuse of data blocks while they are still resident in the faster cache level**

The Locality Principle

- The *locality principle* is the **driving force** that makes the memory hierarchy (caches) work properly. It increases the probability of reusing data blocks that were previously moved from level n to level $n-1$ (i.e., closer to the CPU), thus reducing the miss rate.
- **Temporal Locality (data reuse)**: Temporal locality refers to the property of a program to repeatedly access the same memory locations over a short period of time
 - **Property of the data access pattern**; the cache mapping strategy (direct cache vs associative cache) and the replacement algorithm (LRU, Random) have an impact on temporal locality
- **Spatial Locality**: Spatial locality refers to the property of a program to access memory locations that are spatially close to each other
 - **Enforced by moving data in blocks** between levels of the memory hierarchy (cache line, page)
 - The typical cache line size is **64 bytes** (i.e., it stores 8 – 16 contiguous memory words)

```
//sum and A are global variables
for (int i = 0; i<N; ++i)
    sum += A[i];
```

- both temporal and spatial locality for the instructions, each fetched N times
- `sum` exhibits temporal locality
- `A` is stored in row-major-order `A[0], A[1], ...`
`A` exhibits spatial locality

Terminology

- If the data the processor requests is present in one block at the closest memory level, it is called a **hit**. Otherwise, it is a **miss**, and the next memory level is accessed to retrieve the block containing the requested data
 - **Cache hit**: The data is present in the cache
 - **Cach miss**: The data is not present in the cache
 - **Miss penalty**: the time spent transferring a cache line into the first level cache and the requested data to the processor
 - **Miss rate (MR)**: the fraction of memory accesses not found in the cache $MR = \frac{\# \text{ misses}}{\# \text{ of memory accesses}}$
 - **Hit rate (HR)**: $HR = 1 - MR$

Intermezzo – Measuring CPU_{time} with caches

$$CPU_{time} = ClockCycles \cdot ClockCycleTime = IC \cdot CPI \cdot ClockCycleTime$$

- IC (*Instruction Count*) is the number of program instructions executed
- IC can be decomposed as $IC = IC_{CPU} + IC_{MEM}$, where IC_{CPU} is the number of ALU instructions (e.g., register - register), and IC_{MEM} is the number of memory access instructions (e.g., load, store)
- CPI is the average *ClockCycles Per Instruction*, and is defined as $\frac{ClockCycles}{IC}$
 - Again $CPI = \left(\frac{IC_{CPU}}{IC}\right) \cdot CPI_{CPU} + \left(\frac{IC_{MEM}}{IC}\right) \cdot CPI_{MEM}$, where CPI_{CPU} is the average cycles per ALU instruction and CPI_{MEM} is the average cycles per memory instruction
 - Considering that each memory instruction may generate a cache hit or miss with a given probability, and given the *HitRate* the probability of a cache hit, we have:

$$CPI_{MEM} = HitRate \cdot CPI_{MEM-HIT} + (1 - HitRate) \cdot CPI_{MEM-MISS}$$

- $CPI_{MEM-HIT}$: cycles for a cache hit (typically 1-2 cycles)
- $CPI_{MEM-MISS}$: cycles for a cache miss (50-200 cycles for LLC miss, i.e., DRAM access)
- $(1 - HitRate)$: probability of miss, i.e., MissRate

Impact of data locality on the CPU time

- Example: Let's consider the following scenario:
 $IC = 10^{11}$, $IC_{MEM} = 2 \cdot 10^{10}$,
 $CPI_{CPU} = 1$, $CPI_{MEM-HIT} = 1$, $CPI_{MEM-MISS} = 100$
 $CPU_{freq} = 2GHz$

Question: **What is the impact on the CPU_{time} if the cache hit rate ($HitRate$) drops from 95% to 60%?**

- $CPU_{time}(sec) = IC \cdot \left[\left(\frac{IC_{CPU}}{IC} \right) \cdot CPI_{CPU} + \left(\frac{IC_{MEM}}{IC} \right) \cdot (HitRate \cdot CPI_{MEM-HIT} + (1 - HitRate) \cdot CPI_{MEM-MISS}) \right] \cdot ClockCycleTime$
 - $CPU_{time}(sec) = 10^{11} \cdot \left[\left(\frac{(10^{11} - 2 \cdot 10^{10})}{10^{11}} \right) \cdot 1 + \left(\frac{2 \cdot 10^{10}}{10^{11}} \right) \cdot (HitRate + (1 - HitRate) \cdot 100) \right] \cdot (0.5 \cdot 10^{-9})$
- $\frac{CPU_{time}(HitRate=0.6)}{CPU_{time}(HitRate=0.95)} = \frac{446 \text{ sec}}{99.5 \text{ sec}} = 4.48$
- A decrease in the cache hit rate from 95% to 60% (about 37%) resulted in a **4.48× increase** in execution time!
- It is therefore critical to keep the cache hit rate high to maximize performance
- Key takeaway:** Cache performance has a multiplicative (not additive) impact on execution time.

Cache Algorithms

1/4

- In all general-purpose architectures the cache hierarchy is not explicitly managed by the user
 - It is managed by a set of caching policies (*cache algorithms*) that determine which data is cached during program execution, where the data is stored, and what cache line should be evicted if the cache is full
 - Additionally, the *cache coherence* algorithm keeps data coherent if there are multiple caches

Important questions:

- Which data should we load from main memory?
- Where should we store it in the cache?
- If the cache is full, what data should we evict?

Cache Algorithms

2/4

- **Cache Line:** several items of information (memory words) are stored as a single memory location to enforce **spatial locality**
- Rather than requesting a single value, an entire cache line is loaded with values from neighboring addresses
- Example (**cache line size of 64B**, double precision values):
 - First iteration: $a[0]$ is requested resulting in a **cache miss**
 - Eight consecutive values $a[0]$, $a[1]$, $a[2]$, $a[3]$, $a[4]$, $a[5]$, $a[6]$, $a[7]$ loaded into the same cache line (transfer from memory happens at granularity of cache lines - the block factor is 8)
 - Thus the next seven iterations will then result in **cache hits**
 - Subsequent request $a[8]$ results again in a cache miss, and so on
 - Overall, the hit ratio in our example is as high as 87.5% (7/8)

Important questions:

- Which data should we load from main memory?
- Where should we store it in the cache?
- If the cache is full, what data should we evict?

```
//maximum of an array
//(elements stored contiguously)
for (i = 0; i < n; i++)
    maximum = max(a[i], maximum);
```

$$MR = \frac{n/b}{n} = \frac{n/8}{n} = \frac{1}{8}$$

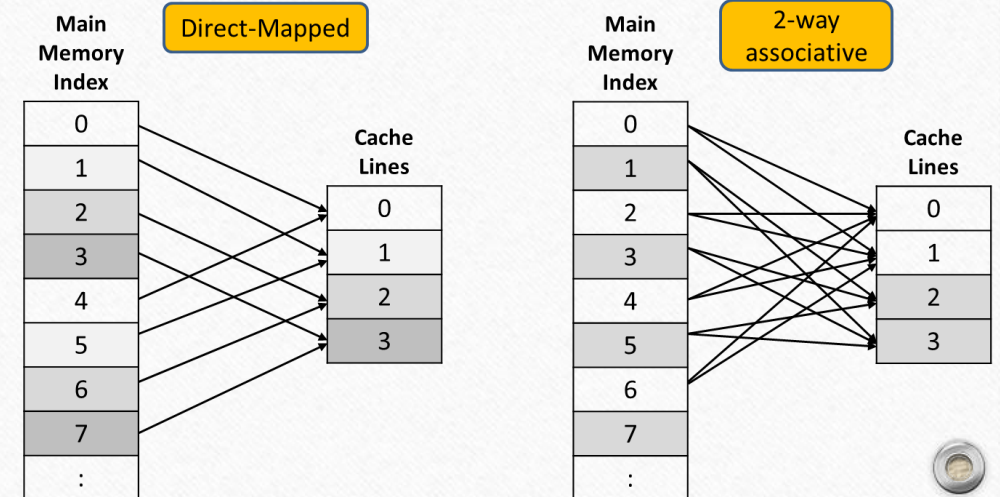
Cache Algorithms

3/4

- Cache organized into a number of **cache lines**
- Cache mapping strategy decides in which location in the cache a copy of a particular entry of main memory will be stored
- **Direct-Mapped Cache**: Each memory block is restricted to exactly one cache line (high miss rates, **thrashing problem**, no temporal locality for cache line replacement algorithm)
- **N-way Set Associative Cache**: Each memory block can be placed in any of N lines within a set, balancing hardware complexity with reduced conflict misses
- **Fully Associative Cache**: Any memory block can occupy any cache line, offering maximum flexibility but at the cost of higher complexity

Important questions:

- Which data should we load from main memory?
- Where should we store it in the cache?
- If the cache is full, what data should we evict?



Cache Algorithms

4/4

- **Cache replacement algorithms** are used to efficiently manage the limited cache space
- When the cache is full, the algorithm decides which cache line to evict to make room for the new cache line that contains the requested data
 - Algorithms: LRU, LFU, FIFO, Random, Pseudo-LRU, ...
- **Least Recently Used (LRU)**: It is an eviction policy used to select one cache line to remove **according to the temporal locality principle**. It evicts the least recently accessed cache line
 - **Pros**: Optimize workloads with temporal locality
 - **Cons**: High overhead due to the access tracking
- **Pseudo-LRU (PLRU)**: An approximation to the LRU algorithm that reduces hardware overhead and complexity. It uses a small set of bits to keep track of which ways within a set have been recently accessed

Important questions:

- Which data should we load from main memory?
- Where should we store it in the cache?
- If the cache is full, what data should we evict?

Cache Working Set

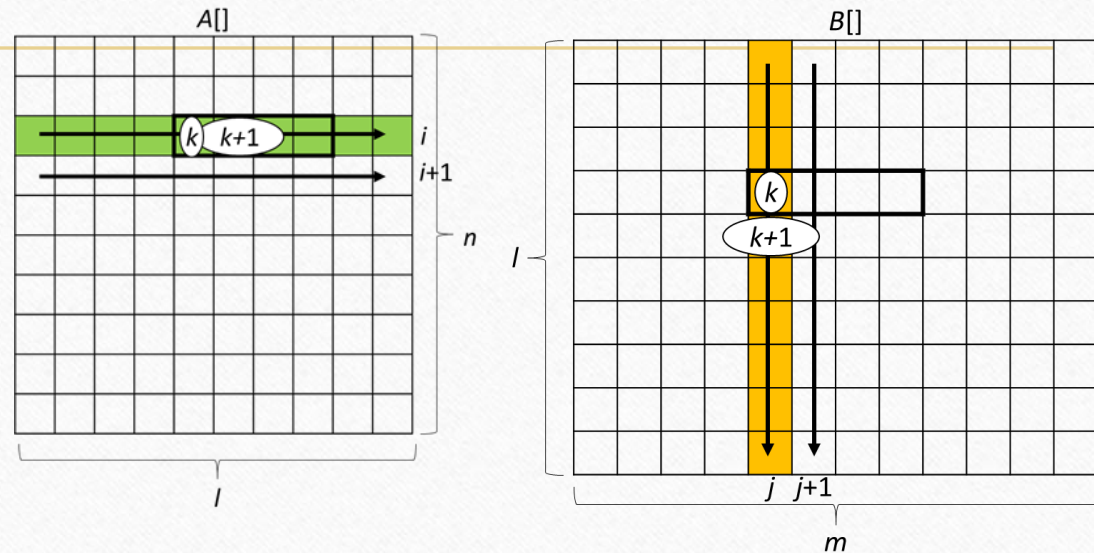
- The *working set* (WS) of a program, for a given a memory hierarchy, **is the collection of data the program actively accesses during a specific time interval**
- Example: Consider a memory hierarchy with levels M1 (e.g., cache) and M2 (e.g., main memory), and a program P . If the entire working set of P fits in M1, the probability of cache misses is minimized
- Important points:
 - A program's working set usually changes over time as it accesses different parts of its data
 - The size and composition of the working set depend on the program's memory access patterns (e.g., sequential vs. random access).

Optimizing Cache Accesses: GEMM

1/7

```
//Matrix Multiplication (textbook)
for (int i = 0; i < n; ++i)
  for (int j = 0; j < m; ++j) {
    float accum = 0.0f;
    for (k = 0; k < l; ++k)
      accum += A[i*l + k]*B[k*m + j];
    C[i*m + j] = accum;
  }
```

GEMM stands for General Matrix Multiply

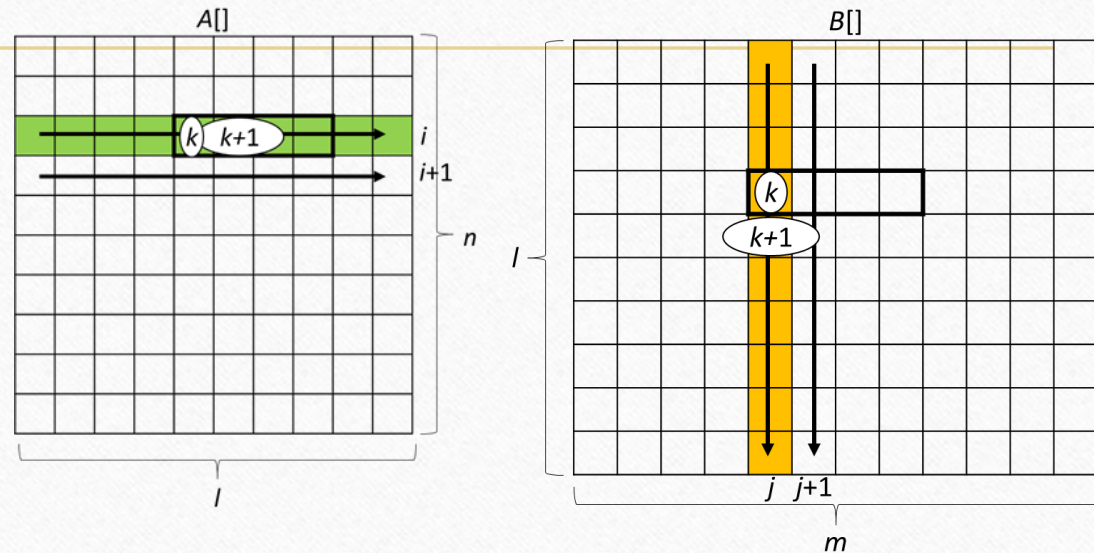


- Matrix Multiplication **textbook algorithm** (naïve): $C_{n \times m} = A_{n \times l} \times B_{l \times m}$ stored in linear arrays in **row-major order**
- Access pattern:
 - The matrix A is accessed contiguously $(i, k) \Rightarrow (i, k+1)$. The i -th row of A is accessed m times.
 - The matrix B is accessed non-contiguously $(k, j) \Rightarrow (k+1, j)$. The entire matrix B is accessed n times.
 - Consecutive B elements are $l \times \text{sizeof}(\text{float})$ bytes apart, **thus they are not stored in the same cache line for large l**
 - Cache lines containing B elements can be evicted from L1/L2-cache before they are reused** in subsequent iteration of j for large l

Optimizing Cache Accesses: GEMM

2/7

```
//Matrix Multiplication (textbook)
for (int i = 0; i < n; ++i)
  for (int j = 0; j < m; ++j) {
    float accum = 0.0f;
    for (k = 0; k < l; ++k)
      accum += A[i*l + k]*B[k*m + j];
    C[i*m + j] = accum;
  }
```

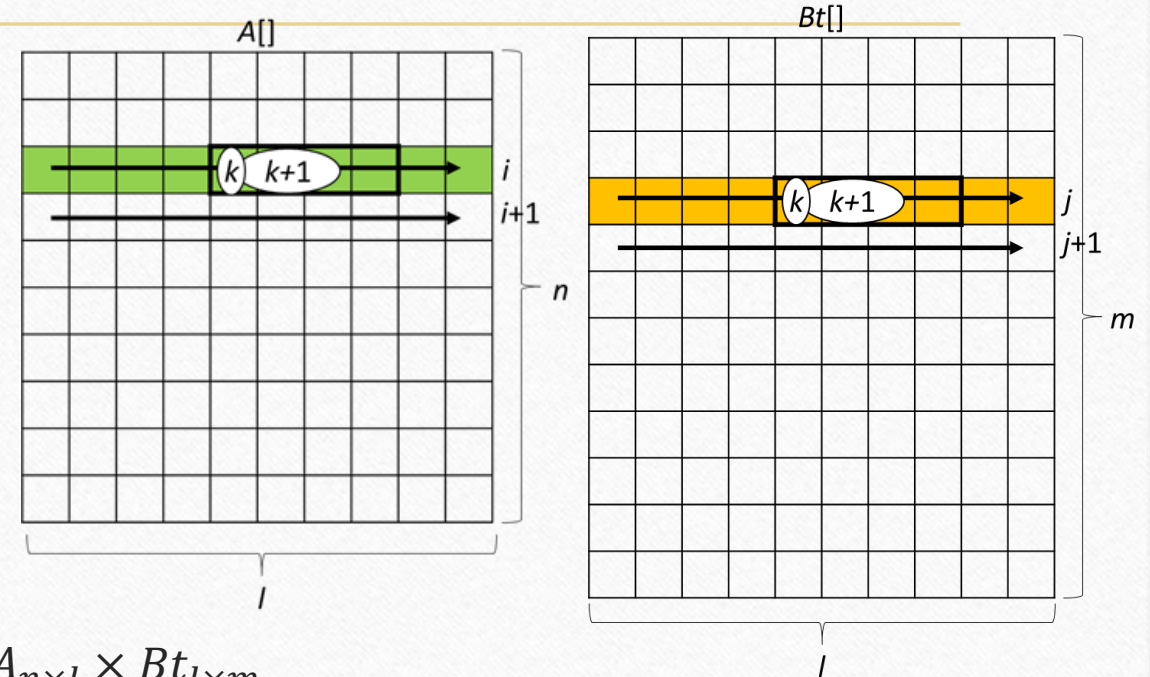


- The working set of the textbook GEMM algorithm is:
 - One row of A, one column of B and one element of C. This is the minimum working set that minimizes cache misses for a relatively small time interval (considering innermost loop)
 - However, if we consider a large enough time interval (i.e., considering the for j loop), the working set is one row of A, one row of C and the entire matrix B
 - For large l and m , B will not fit into LLC. What we can do is to improve cache locality by exploiting at least spatial locality for B

Optimizing Cache Accesses: GEMM

3/7

```
//Transpose-and-Multiply
for (int k=0; k<l; ++k)
  for (int j = 0; j<m; ++j)
    Bt[j*l + k] = B[k*m + j];
for (int i=0 ; i<n; ++i)
  for (int j=0; j<m; j++) {
    float accum = 0.0f;
    for (int k=0; k<l; ++k)
      accum += A[i*l + k] * Bt[j*l + k];
    C[i*m + j] = accum;
  }
```



- **Transpose-and-Multiply:** $Bt_{m \times l} = (B_{l \times m})^T$ and $C_{n \times m} = A_{n \times l} \times Bt_{l \times m}$
- Access pattern of A contiguously: $(i, k) \Rightarrow (i, k+1)$
- Accesses pattern of Bt contiguously: $(j, k) \Rightarrow (j, k+1)$
- This new algorithm leverages spatial locality for A , and B (i.e., Bt)
- It introduces an extra overhead for transposing B

Optimizing Cache Accesses: GEMM

4/7

Execution on an i7-6800K using $m = n = l = 2^{13}$

#elapsed time (naive_mult):	5559.5s
#elapsed time (transpose):	0.8s
#elapsed time (transpose_mult):	497.9s

Speedup: 11.1

Execution on an i7-6800K using $m = n = 2^{13}, l = 2^8$

#elapsed time (naive_mult):	28.1s
#elapsed time (transpose):	0.01s
#elapsed time (transpose_mult):	12.9s

Speedup: 2.2

spmcluster (front-end)
Intel(R) Xeon(R) CPU
E5-2650 v3 @ 2.30GHz

```
[torquati@spmln spmcode1]$ make mm
g++ -std=c++17 -I. -I./include -Wall -O3 -march=native -o mm mm.cpp
[torquati@spmln spmcode1]$ ./mm 13 13 8
# elapsed time (naive_mult): 41.5855s
# elapsed time (transpose): 0.00975489s
# elapsed time (transpose_mult): 17.5704s
# elapsed time (transpose_and_mult): 17.5802s
```

Speedup: 2.36

Optimizing Cache Accesses: GEMM

5/7

```
// initialize C to 0.0f
std::fill(C, C + n*m, 0.0f);
//Matrix Multiplication (loop order: ikj)
for (int i = 0; i < n; ++i)
    for (int k = 0; k < l; ++k) {
        float Aik = A[i*l + k]; // reuse across j
        for (int j = 0; j < m; ++j)
            C[i*m + j] += Aik*B[k*m+j];
    }
```

Execution on the spmcluster using $m = n = 2^{13}, l = 2^{11}$

#elapsed time (transpose_and_mult): 168.8s

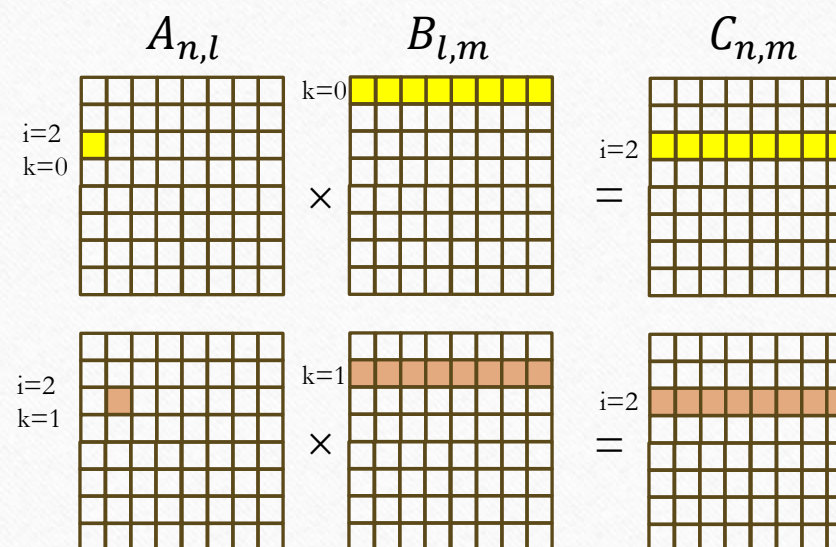
#elapsed time (loop_interchange): 53.85s

Speedup: 3.13

The textbook version takes about 1412s.

The speedup considering the loop interchange optimization is 26.22×

- Loop interchange optimization (loop order ikj)
 - Better locality: reuse $A[i,k]$ across j while accessing $B[k,]$ and $C[i,]$ contiguously with unit stride

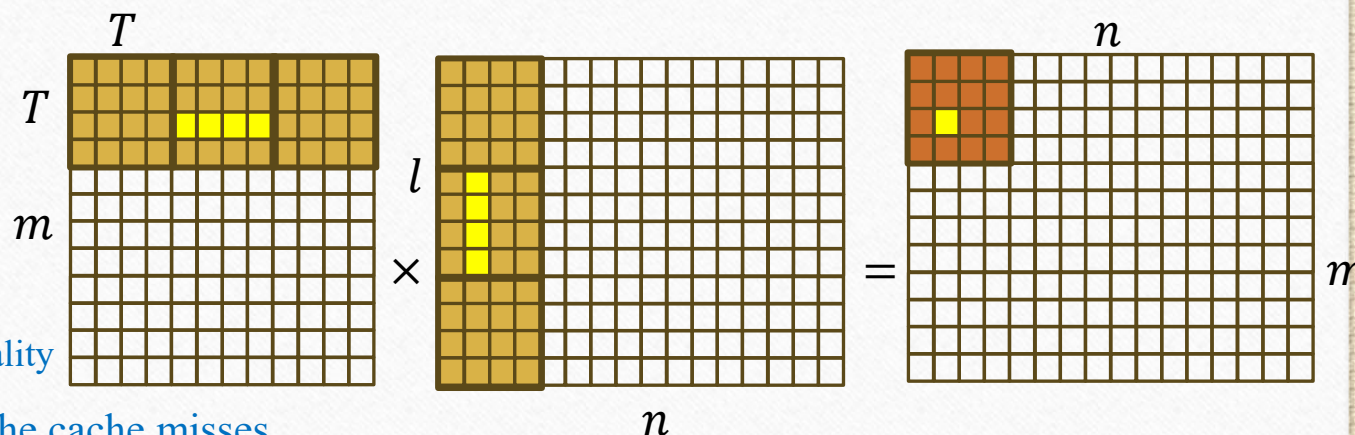


Optimizing Cache Accesses: GEMM

6/7

- How can we reduce the working set to fit into the cache?
- We can **change the algorithm** and use a block-based (more often called **tile-based**) GEMM

- Instead of working on the entire matrix at once, we can split it into smaller sub-matrices (tiles) of size $T \times T$
- The working set consists of a tile from each matrix (A, B, C)
 - These tiles exhibit both spatial and temporal locality
- If the three tiles fit into the cache we minimize the cache misses



T should be chosen such that 3 tiles fit in the cache of size *Cachesize*.

The memory needed is: $3 \cdot T^2 \cdot \text{sizeof}(\text{data})$, therefore we can approximate T as
Typical values range from 16 to 128. **Start with $T=16$ than $T=32$, etc... and measure the time!** Modern libraries (MKL, OpenBLAS) use auto-tuning to find the best T

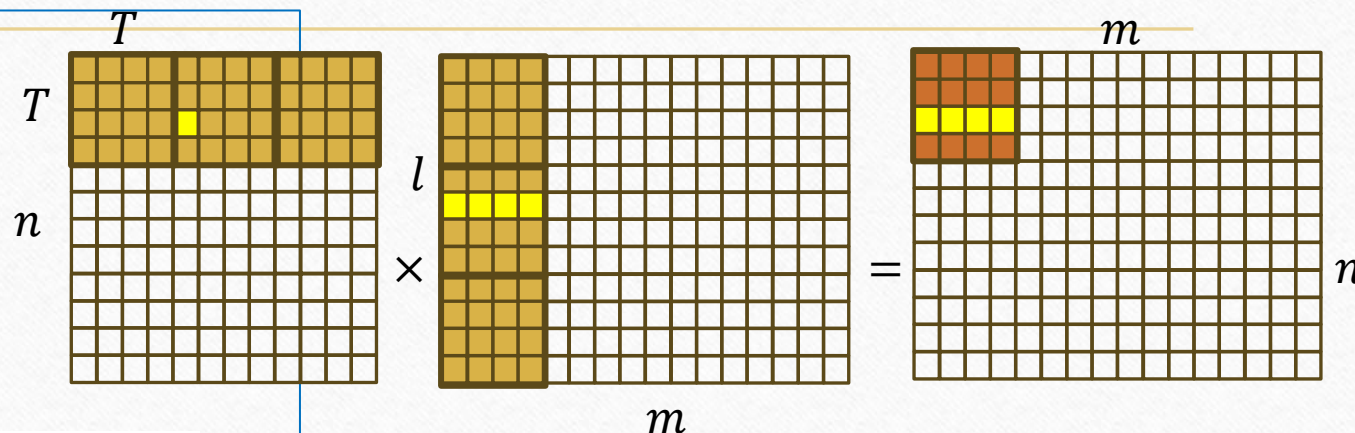
$$T \cong \left\lceil \sqrt{\frac{\text{Cachesize}}{3 \cdot \text{sizeof}(\text{data})}} \right\rceil$$

Optimizing Cache Accesses: GEMM

7/7

```
// initialize C to 0.0f
std::fill(C, C + n*m, 0.0f);
//Tile-based Matrix Multiplication (MM)
for (int i = 0; i < n; i += T)
  for (int k = 0; k < l; k += T)
    for (int j = 0; j < m; j += T) {

      // Compute MM for the current tile
      for (int ii = i; ii < std::min(i + T,n); ++ii)
        for (int kk = k; kk < std::min(k + T,l); ++kk) {
          float a = A[ii * l + kk]; // reuse across jj
          for (int jj = j; jj < std::min(j + T,m); ++jj)
            C[ii * m + jj] += a * B[kk * m + jj];
        }
    }
}
```

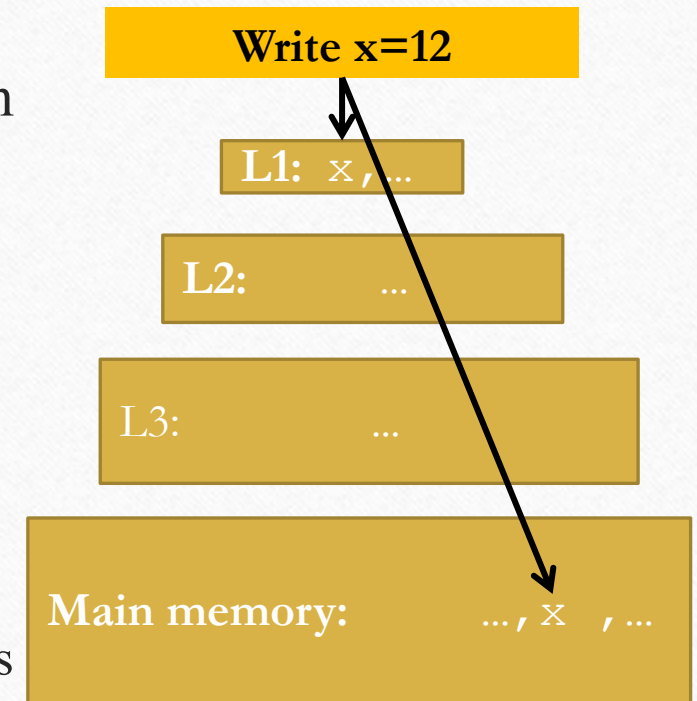


Execution on the spmcluster using $m = n = 2^{13}$, $l = 2^{11}$	
T=64	41.69
T=96	36.25
T=104	34.29
T=124	30.87s
T=128	34.14s

You should always profile and tune your code on the target machine!

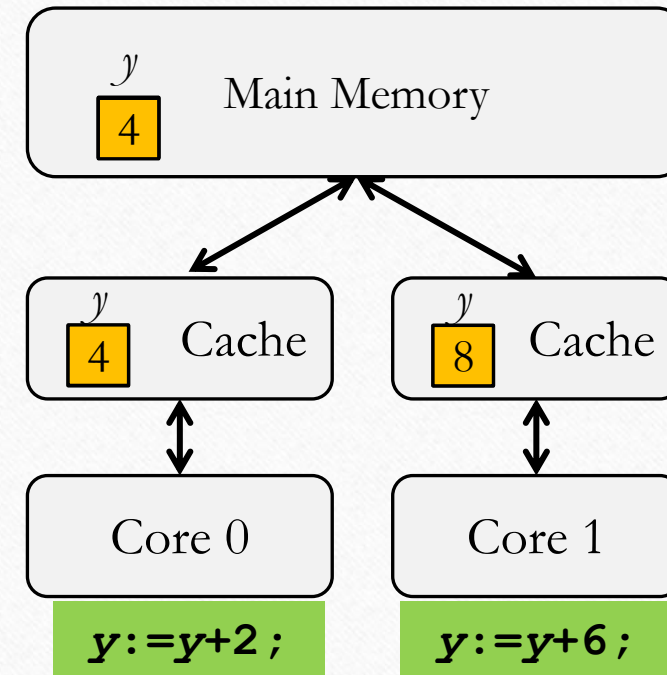
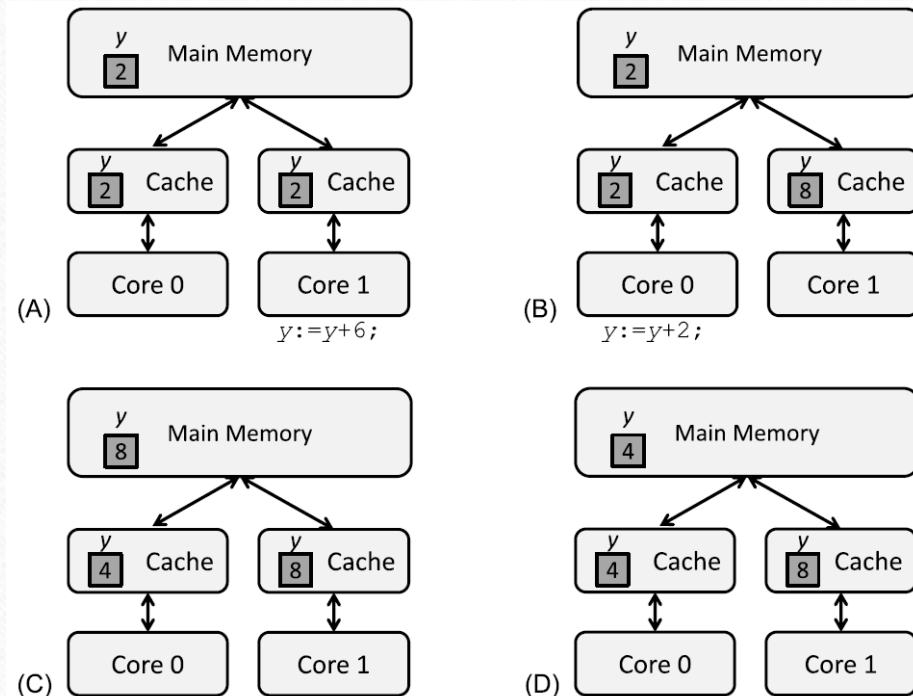
Cache Write Policies

- When a CPU writes data to the cache, the value in the cache may be **inconsistent (the data is not coherent)** with the value in main memory.
- **Write-through policy**
 - Caches handle this by updating the data in the main memory when it is written to the cache. Always implemented with a **store/write buffer**
- **Write-back policy**
 - Caches mark data in the cache as dirty (one extra bit per cache line needed – dirty bit). More complex/costly to realize.
 - Modified data (dirty bit = 1) written to memory only when cache line is evicted from the cache
 - A **store/write buffer** is generally used to reduce the cost of cache accesses



The Cache Coherence Problem

- With private caches per core, it is possible to have several copies of shared data in distinct caches, each cache stores a different value for a single address location



Incoherent Caches: the two caches store different values for the same variable

Cache Coherence Mechanisms

- Cache coherence protocol ensures that multiple processors accessing the same memory location see a consistent view of the data. Two primary mechanisms:
- **Update-based**: it propagates updates to all cache copies when a processor modifies a cached data
 - Best for: read-intensive workloads
- **Invalidation-based**: when a processor modifies a cache line, all other copies in other caches are invalidated instead of being updated
 - An invalidated cache must refetch data (from another cache or main memory)
 - Best for: write-intensive workloads
- Update-based protocols require more bandwidth but may reduce cache misses
- Invalidation-based protocols require less bandwidth but exchange more messages, and are easier to implement

Example: Initially, $x=0$
Core0 writes $x=1$, Core1 reads x

Update-based:

- Core0 writes $x=1$ and broadcasts the new value to all caches
- Core1's cache updated to $x=1$
- Next read by Core1: immediate HIT with correct value

Invalidation-based (e.g., MESI/MOESI):

- Core0 writes $x=1$ and broadcasts the invalidation signal
- Core1's cache line marked INVALID
- Next read by Core1: MISS, must fetch $x=1$ from Core0/Memory

Cache Coherence: MESI

- Cache coherent protocol

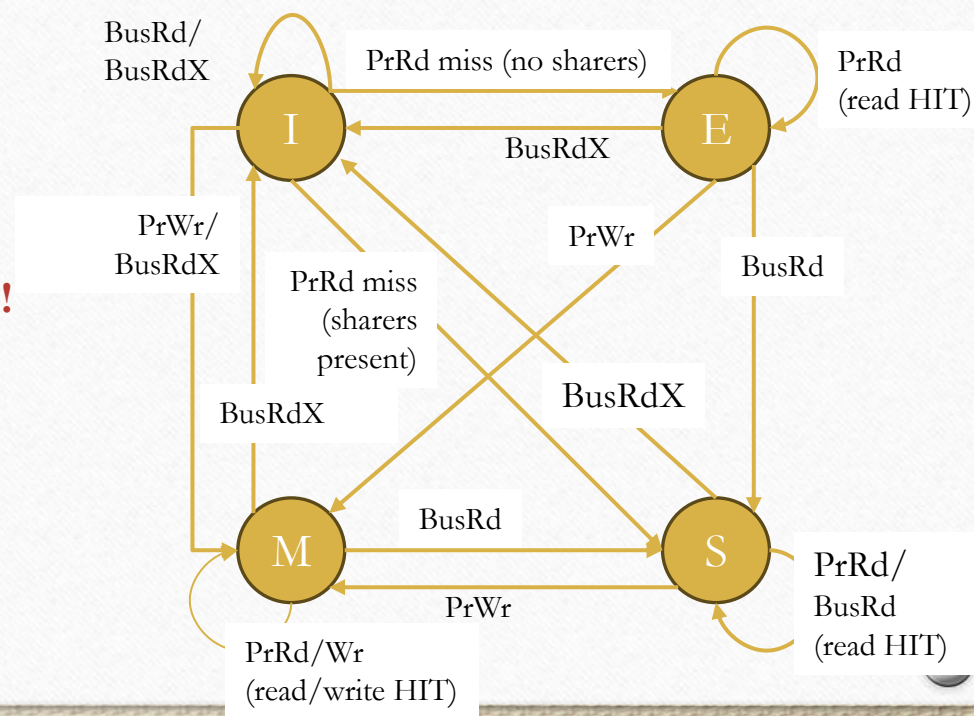
- A cache coherence protocol is **implemented at hardware level**
- Coherence ensures **single-writer / multiple-reader semantics**
- A prominent example is the **MESI protocol** (**invalidation-based protocol**), which marks a cache line using four possible states: Modified (**M**), Exclusive (**E**), Shared (**S**), and Invalid (**I**)
 - The **MOESI protocol** introduces an additional state: Owned (**O**)
- The granularity of the cache coherency protocol is the cache line!**
- Remember: all modern general-purpose CMPs provide **automatic HW-based cache coherence protocols**
 - Some GPUs and specialized accelerators do not guarantee automatic cache coherence across all levels

Snoopy bus implementation of MESI.

Viewpoint of one core (**local processor Pr**) for one cache line that can be read (**Rd**) and written (**Wr**).

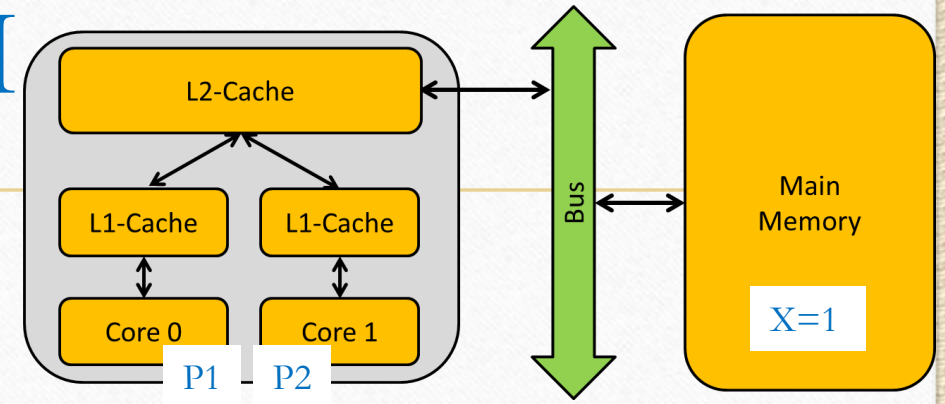
PrRd and **PrWr** are local read and write events.

BusRd read request observed on the bus, typically due to a read miss in another core. **BusRdX** request for eXclusive ownership, typically triggered by a write miss in another core



Cache Coherence: MOESI

- MOESI is a write-invalidate protocol that extends MESI with the **Owned (O)** state
- When another core reads a line that is modified in a cache, the data can be supplied cache-to-cache, and the owner transitions to **O**, so memory is not updated until a later **write-back** (e.g., on eviction).
- The simplest implementation uses a *snooping bus protocol* (more advanced implementations use a directory-based protocol)



Time	Event	Bus activity	L1-core0	L1-core1	Mem
					X=1
	P1 reads X	Miss for X (load from Mem)	X=1 (E xclusive)		X=1
	P2 reads X	Miss for X (load from P1's cache)	X=1 (S hared)	X=1 (S hared)	X=1
	P1 writes X	Invalidate X in other caches	X=0 (M odified)	X=? (I nvalid)	X=1
	P2 reads X	Miss for X (load from P1's cache)	X=0 (O wned)	X=0 (S hared)	X=1

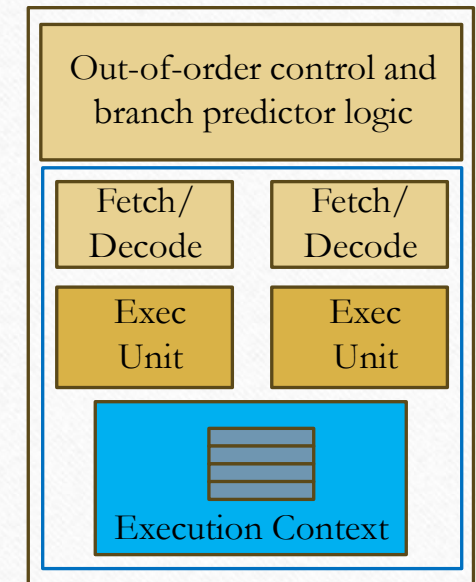
Example using **MOESI** and **write-back** caches

O stands for **Owned**, the core has a dirty shared copy of the line, and it is responsible for supplying data.

In the **write-back** cache, memory is stale until eviction of the cache line

Superscalar processors

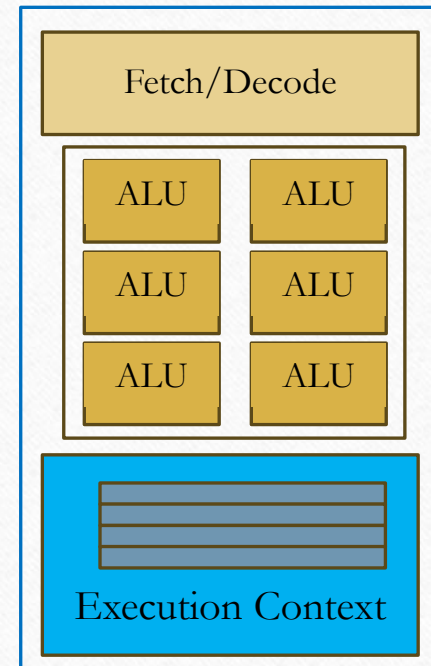
- Modern CPUs are highly parallel (and quite complex), mixing **pipelining** and **superscalar** technologies
- Superscalar CPUs were designed to **execute multiple instructions from a single process/thread** (ILP - Instruction Level Parallelism) **simultaneously** to improve performance and CPU utilization
- The processor **fetches and issues multiple instructions concurrently in a single clock cycle**
- **Executes them out-of-order** (i.e., as soon as operands are available), resolving data hazards (RAW, WAW, WAR) via register renaming, to maximize utilization of execution units while respecting data dependencies
- Results are then re-ordered (through a reorder buffer – ROB) to ensure they are written back to the register file or memory in the correct program order
- Finally, instructions are **committed (retired) in program order or discarded (flushed)** due to branch misprediction together with any dependent instructions



Execution Context is the complete processor state for one thread (PC, Register file, processor status flags)

Vector Units

- To increase compute capability, execution units typically contain one or more **vector units** that can process multiple data elements in parallel
- The point is to execute a Single Instruction Stream on multiple data (**SIMD execution model**)
- The vector unit contains multiple ALUs, each processing a different **lane** of a **vector register** that is wider than standard scalar registers
- The same instruction is broadcast to all ALUs
- Each ALU computes the operation in parallel on different data



Simple example:

Vector addition $c[i] = a[i] + b[i]$

Scalar execution (4 iterations):

$c[0] = a[0] + b[0] // 1 \text{ cycle}$

$c[1] = a[1] + b[1] // 1 \text{ cycle}$

$c[2] = a[2] + b[2] // 1 \text{ cycle}$

$c[3] = a[3] + b[3] // 1 \text{ cycle}$

Total: 4 cycles

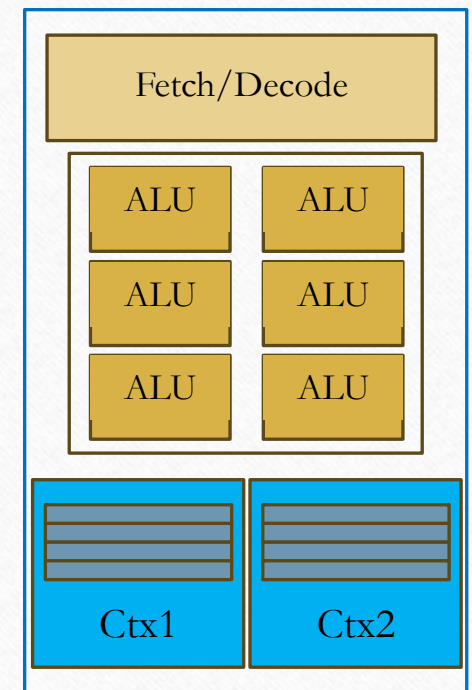
SIMD execution (256-bit register):

$c[0:3] = a[0:3] + b[0:3] // 1 \text{ cycle}$

Total: 1 cycle (4× speedup)

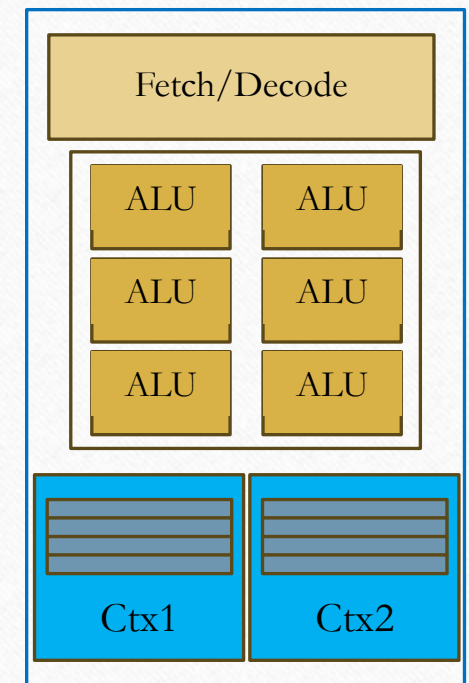
Multiple threads/contexts of execution

- Pipeline and superscalar processors were not enough
 - Both techniques improve performance. Pipelining overlaps the execution of different stages, allowing a new instruction to start at every cycle. Superscalar execution allows multiple instructions to be issued and executed in parallel at each cycle, exploiting ILP within a thread
- **Sequential bottleneck:** Even in well-optimized sequential code, **the number of truly independent instructions within a single thread is limited**, bounding the exploitable ILP
 - True data dependencies (RAW hazards), false dependencies (WAW, WAR), and control dependencies (branches) all reduce the pool of independent instructions
 - All this plus cache misses stalls the processors and idles the pipeline if there aren't enough ready-to-run instructions in a single thread
- To overcome such low efficiency, **HW Multi-Threading** has been added in superscalar processors to execute multiple instructions **from multiple threads of control simultaneously**
 - Each thread has its own set of registers and program counter (PC) and processor status flags, **representing its execution context**. The processor maintains the context of each thread to quickly switch between them.
 - Typically, 2-4 contexts per core (IBM Power10 has 8 contexts per core)



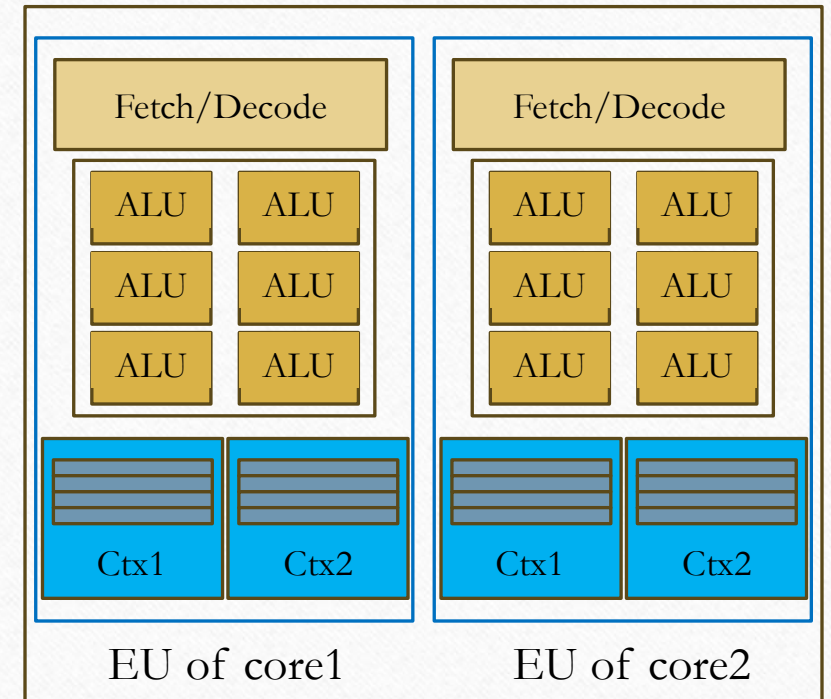
HW Multi-Threading

- HW Multi-threading (or multithreading) enables a single core to execute multiple threads concurrently
 - A thread represents an independent sequence of instructions
- There are three main types of HW multithreading:
 - **Fine-grained Multi-Threading (FMT)** interleaves instructions from different threads one at a time at the instruction level (i.e., switches instructions between threads every cycle)
 - **Coarse-grained Multi-Threading (CMT)** switches between threads at the thread level. A switch happens only when the thread in execution causes a stall (e.g., long-latency events such as L1 cache miss)
 - **Simultaneous Multi-Threading (SMT)** interleaves the execution of instructions from different threads. Instructions are simultaneously issued to the execution units from multiple threads at each clock cycle.
- **Hyperthreading**: Intel's specific implementation of SMT, which optimizes the use of resources
 - For the OS, each context is a logical core



Modern CPU Design

- Modern processors combine **pipelining**, **superscalar execution**, **HW multithreading**, **multi-core**, and **vector units** within each core; all parallelism can be exploited simultaneously
 - Pipelining** increases instruction throughput per core
 - Superscalar** enables multiple instructions per clock cycle in a single core
 - HW Multi-Threading** improves efficiency by minimizing stalls due to memory latency, branch misprediction, and instruction dependencies
 - Multi-Core** allows true parallel execution of multiple application threads across cores
 - Vector Units** enable data parallelism at the instruction level within a core



Forms of Parallel Execution in Modern CPU

- **Instruction Level Parallelism (ILP)**

- Superscalar cores execute multiple instructions from the same instruction streams in parallel within a core
- Hardware dynamically discovers and schedules independent instructions

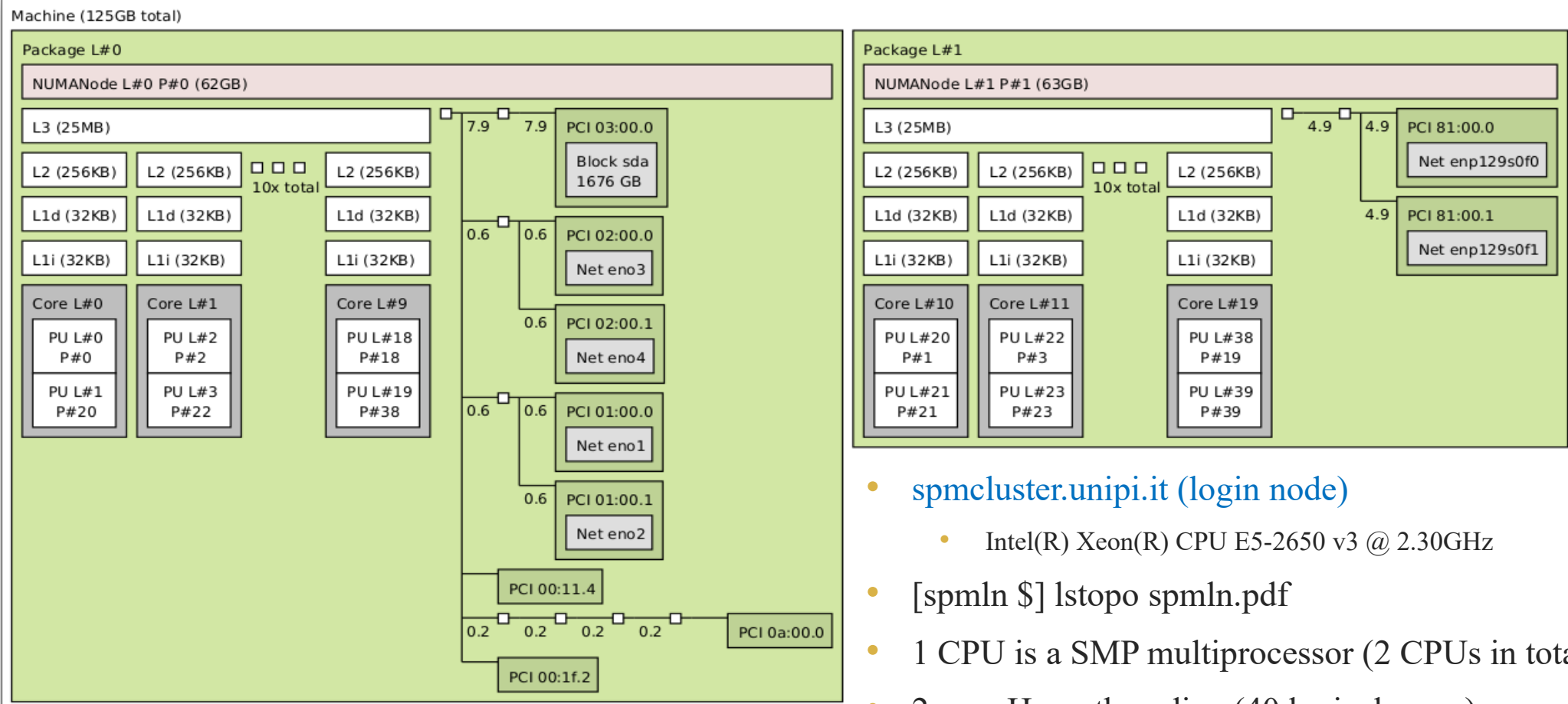
- **SIMD parallelism**

- Cores with vector units execute the same instruction on multiple data elements simultaneously within a core
- Achieved through vectorization by the programmer (explicit vectorization) or by the compiler (implicit vectorization)

- **Multi-Core parallelism**

- Multiple cores execute separate instruction streams independently, enabling true parallelism
- Requires software-managed parallelism through APIs and programming models

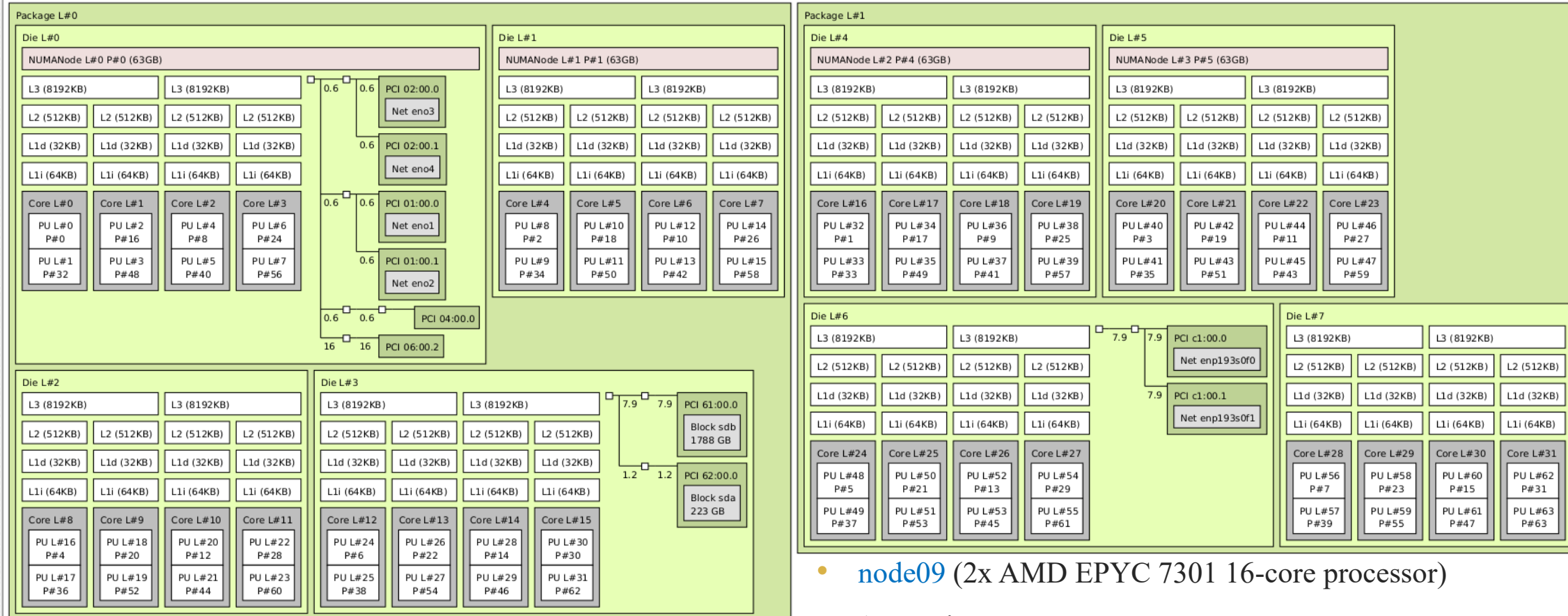
Core/Cache topology using lstopo



- spmcluster.unipi.it (login node)
 - Intel(R) Xeon(R) CPU E5-2650 v3 @ 2.30GHz
- [spmln \$] lstopo spmln.pdf
- 1 CPU is a SMP multiprocessor (2 CPUs in total)
- 2-way Hyperthreading (40 logical cores)

Core/Cache topology using lstopo

Machine (252GB total)

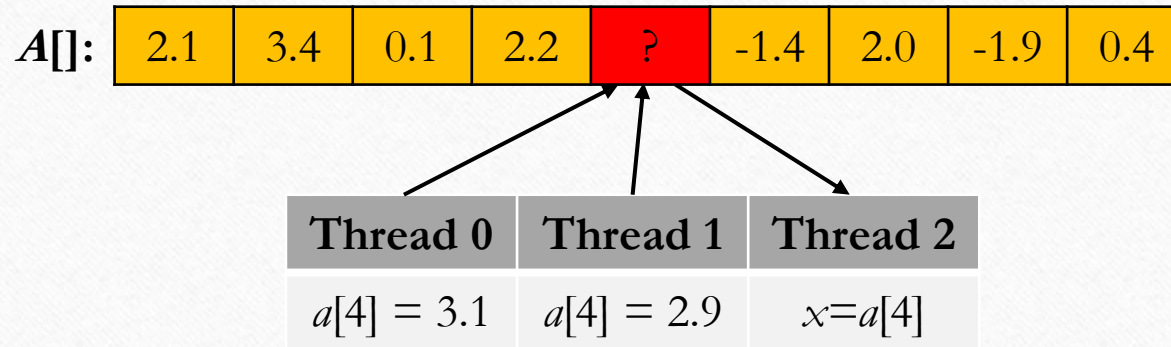


- **node09** (2x AMD EPYC 7301 16-core processor)
- 1 CPU is a NUMA CMP
- 2-way SMT (64 logical cores)

Threading on Multi-Cores

- Parallelism is achieved by **starting threads** that run concurrently on the system
- Thread creation is generally more lightweight and faster compared to process creation:
 - Creating a process is roughly 3-5 times slower than creating a thread
 - In modern systems, creating a thread takes **$O(10^4)$ clock cycles**
 - The *fork* system call requires copying/initializing more data (e.g., page table)
 - See the “launch-benchmark.cpp” code
 - Exact ratios depend on OS and workload
- **Data exchange among threads occurs by reading from and writing to shared memory**
 - Processes data exchange usually requires IPC (i.e., system calls)
 - Memory segment sharing also possible across processes (shared-memory)

The Data Race problem



- **Data Race (DR)**: occurs when two (or more) threads access a shared variable simultaneously and:
 - at least one access is a write operation
 - the accesses to the shared variable are not separated by synchronization operations
- **DRs produce non-deterministic behavior**;
- **Synchronization mechanisms prevent DRs**: only one thread modifies/reads at a time
 - *mutexes, condition variables, semaphores, atomic instructions*

Example, execution timeline:

Time	Thread 0	Thread 1	Thread 2
t0	read $a[4]=3.0$		
t1		read $a[4]=3.0$	
t2	write $a[4]=3.1$		
t3		write $a[4]=2.9$	
t4			read $a[4]=???$

What value does Thread 2 read?

- Could be 3.1 (if Thread 0's write is last)
- Could be 2.9 (if Thread 1's write is last)
- Could be 3.0 (if read happens before writes)

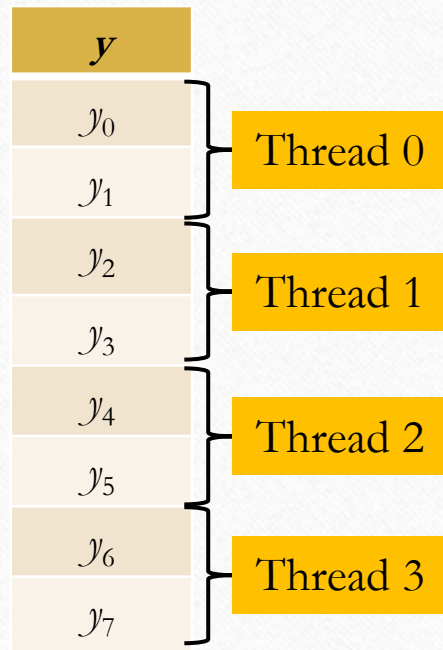
Result: **non-deterministic behavior**!

The False Sharing problem

- Caches are organized in cache lines, where each line holds several values (b words)
 - 8-16 memory words are the typical values of a cache line of 64 bytes
- The cache coherence protocol works by invalidating/updating the entire cache line and not single values within the cache line
- If **distinct** data items (e.g., variables) present in the same cache line are updated by different threads, we have the **false sharing** effect (or *cache line ping-pong* due to the cache coherence).
 - No true sharing actually exists since threads work on different variables!
- The overhead introduced by the cache coherence protocol to keep the cache line coherent among all copies in the caches can be significant
 - This is an indirect effect produced by the coherence protocol that operates at cache line granularity
 - The lines are seen as “Shared” even if the threads are modifying different variables

False Sharing - Example

```
for (int i = 0; i < m; i++) {  
    y[i] = 0.0;  
    for (int j = 0; j < n; j++)  
        y[i] += A[i][j] * x[j];  
}
```



- Consider $m = 8$, and a cache line size of 64 B
 - y fits exactly in **one cache line**
- 4 threads, one per core, each thread computing 2 elements of y
- **False Sharing**: Every write to y invalidates the cache line in the other core's cache. Most of these updates to y force main memory accesses
- **Solution** (two main options):
 - data padding: ensures each thread's slice lay on a separate cache line
 - use temporary variables: reduces the frequency of writes

False Sharing - Example

```
for (int i = 0; i < m; i++) {  
    y[i] = 0.0;  
    for (int j = 0; j < n; j++)  
        y[i] += A[i][j] * x[j];  
}
```

Using local variables
(stored in registers)



```
for (int i = 0; i < m; i++) {  
    float _y = 0.0;  
    for (int j = 0; j < n; j++)  
        _y += A[i][j] * x[j];  
    y[i] = _y;  
}
```

```
// 128-bit packed data that fits  
// into a cache line of 64 B  
// 2 threads, one modify ying  
// the other thread modify yang  
struct pack_t {  
    uint64_t ying;  
    uint64_t yang;  
};
```

Using explicit
data padding



```
// forcing ying and yang to lay in  
// two distinct cache lines  
struct pack_t {  
    uint64_t ying;  
    char padding[CACHELINE_SIZE_BYTE -  
                sizeof(uint64_t)];  
    uint64_t yang;  
};
```


Characterize performance limits

- Modern SHM architectures are increasingly complex
 - Multiple levels of caches with different latencies and bandwidth, non-uniform memory access, aggressive prefetching and speculative execution, vector and scalar code, ...
- Such complexity makes it difficult to predict performance precisely
 - Theoretical peak performance (e.g., R_{peak}) is an upper bound that cannot be achieved
 - Speedup/Scalability measures how additional cores improve overall performance but does not reveal:
 - how close your applications are to hardware limits
 - Whether your application is bottlenecked by compute or memory
 - Which optimization would have the highest impact
- The **Roofline Model** helps addressing these questions by providing:
 - Visual representation of performance bounds
 - Clear identification of bottlenecks (compute-bound or memory-bound)
 - Guidance for optimization priorities (e.g., by improving data locality or leveraging SIMD instructions)

Roofline model motivations

- Questions: How far is our application from the achievable performance on a given system?
 - The Roofline Model provides an answer to this question by providing achievable performance as a function of operational intensity and identifying the limiting factor for each kernel
- The Roofline model is a visual performance analysis model designed to evaluate the computational performance limits of a system by considering the relationship between **operational intensity**, **memory bandwidth**, and **peak computational performance**
 - It helps identify bottlenecks and prioritize optimizations
- Key assumptions:
 - Focuses on loops (at the steady state) as the major source of computational workload
 - Perfect overlap of computation and memory transfers
 - Uses best-case bandwidth and peak compute performance
- We will describe a simplified and basic version of the roofline model

Roofline model assumptions

1/2

- **Simplified HW view**

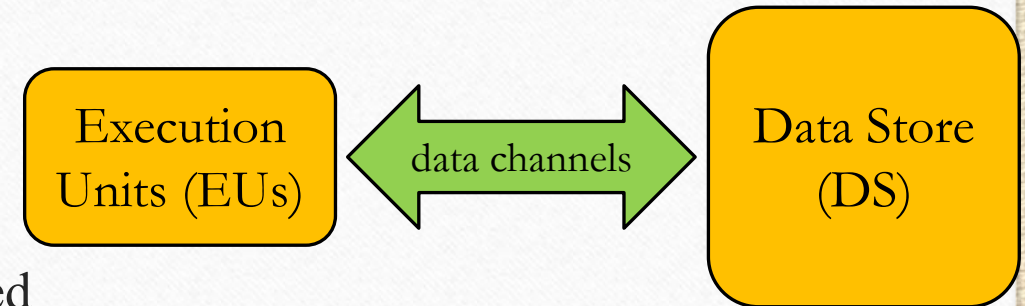
- It abstracts away all complexities of current systems

- Assume all functional units fully utilized
 - No pipeline stalls or idle cycles

- Execution Units (EUs) operate at the maximum speed R_{peak} measured in FLOPS

- The Data Store (DS) is any source of data (DRAM, Cache L1/L2/L3, Disk)

- Data channels to/from DS have a peak bandwidth B (byte/s)



Roofline model assumptions

2/2

- **Simplified HW view**

- Applications contain a sequence of loops with a sufficiently large number of iterations (L) so that transient effects can be ignored
- At the **steady state**, the considered loop **does N FLOP** using **D bytes** of data transferred to/from the DS
- The ratio $I = \frac{N}{D}$ is the **computation intensity** (also called **arithmetic intensity**) measured in **FLOP/byte**
- **Higher I means more computation per byte transferred**

```
for (i = 0; i < L1; i++) {  
    N1 FLOPS over D1 bytes transferred  
}  
...  
for (i = 0; i < L2; i++) {  
    N2 FLOPS over D2 bytes transferred  
}  
...
```

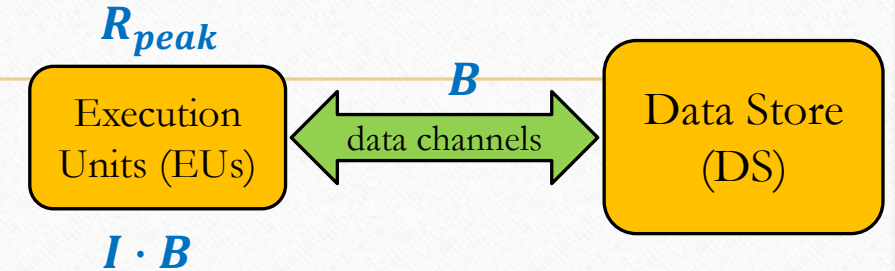
Example:

```
for (i = 0; i < L; i++)  
    c[i] = a[i] + b[i]; // 1 FLOP
```

$$I = \frac{L}{3 \cdot L \cdot 8} = \frac{1}{24} \text{ FLOP/byte}$$

Roofline model overview

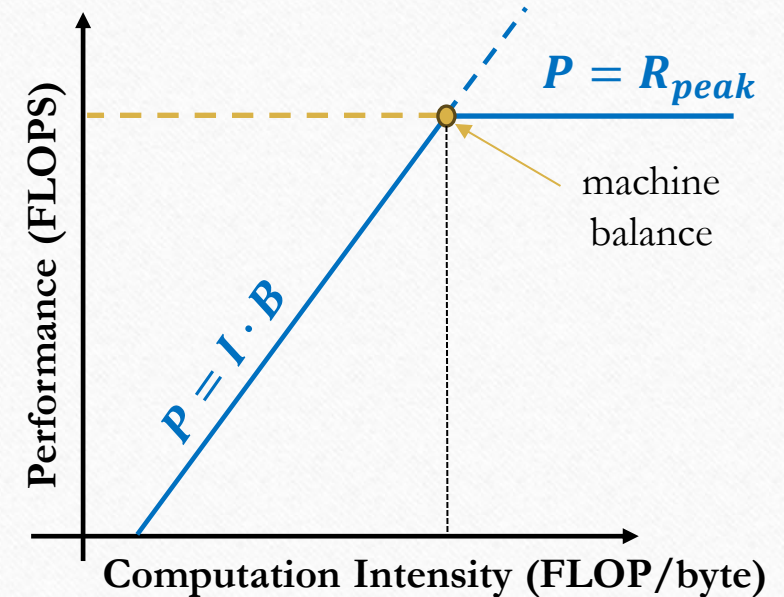
- How quickly can tasks in our application be processed?
What is the expected performance (P) of our application?
- Two possible bottlenecks:
 - **Compute bottleneck:** R_{peak} i.e., the execution of tasks in the EUs (FLOPS)
 - **Memory bottleneck:** B i.e., the data channels' peak bandwidth (byte/s)
- The data channels performance as seen by the EUs can be expressed in FLOPS with the factor $I \cdot B$ ($\frac{FLOP}{byte} \cdot \frac{byte}{s} = FLOPS$)
- R_{peak} and $I \cdot B$ are two upper limits, therefore, the expected performance P is



$$P = \min(R_{peak}, I \cdot B)$$

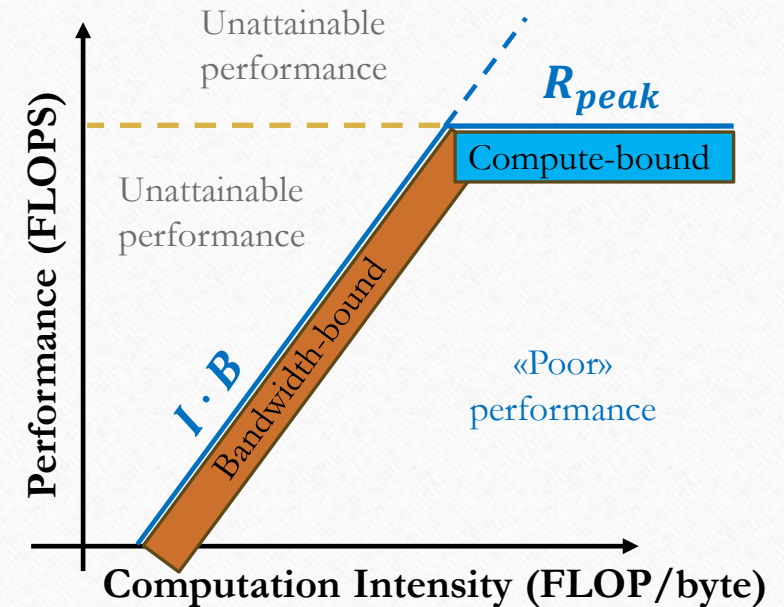
Roofline model overview (cont.)

- $P = \min(R_{peak}, I \cdot B)$
- If the I is low, the performance P is limited by the data transfer B to/from DS (memory/bandwidth-bound)
 - I is interpreted as a measure of the computational quality of code
 - Optimization: reduce data movement, improve cache reuse
- If the I is high, the performance P is limited by the execution of works in the EUs (R_{peak}) (compute-bound)
 - Optimization: vectorization, algorithm improvements
- The best use of resources is the inflection point, called “machine balance”
 - Different architectures have different “machine balance”



Roofline model overview (cont.)

- $P = \min(R_{peak}, I \cdot B)$
- If the I is low, the performance P is limited by the data transfer B to/from DS
 - I is be interpreted as a measure of the computational quality of code
- If the I is high, the performance P is limited by the execution of works in the EUs (R_{peak})
- The best use of resources is the inflection point, called “machine balance”
 - Different architectures have different “machine balance”
- We have 5 regions in the plot: 2 unattainable performance regions, a bandwidth-bound region, a compute-bound region, and a “poor” performance region



Roofline model example

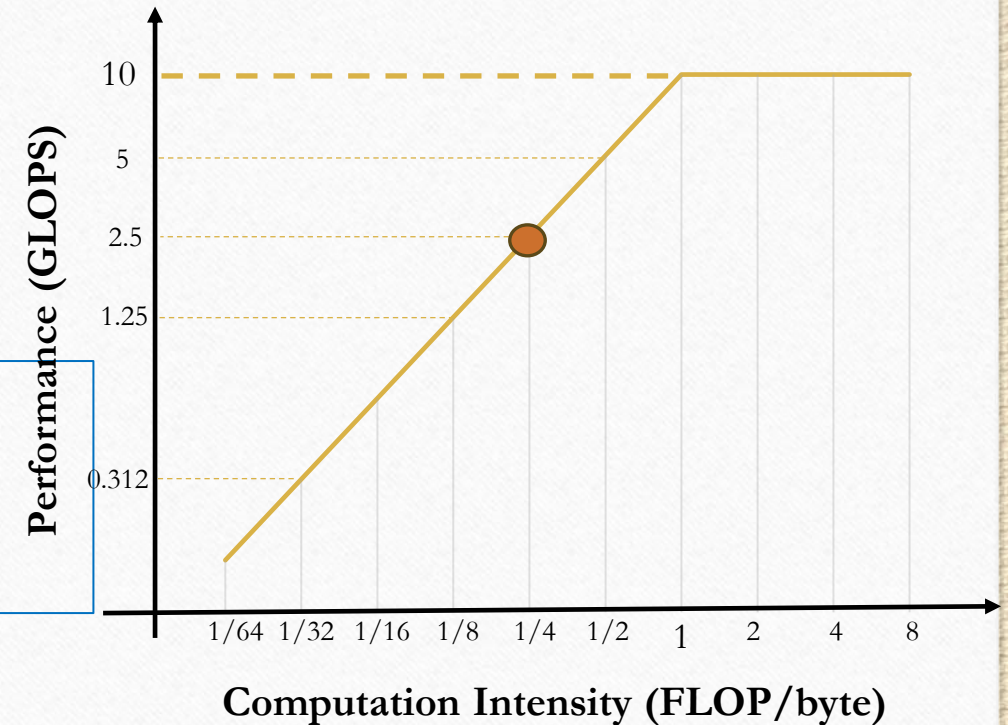
- Let's suppose that in the considered system:

- $R_{peak} = 10$ GFLOPS
- $B = 10$ GB/s

- Let's consider this loop:

```
double s=<some-value>; // s stored in a register
for (int i = 0; i < N; i++) {
    s = s + A[i] * A[i]; // 2 FLOPS
}
```

- $I = \frac{2 \cdot N}{8 \cdot N} = \frac{1}{4} = 0.25$ FLOP/byte at the steady-state
- $P = \min(10, 0.25 \cdot 10) = 2.5$ GFLOPS, which means bandwidth-bound computation (25% of peak)



Roofline model example: DGEMM

- Let's consider a system with $R_{peak} = 64$ GFLOPS and $B = 32$ GiB/s
- Textbook DGEMM ($N \times N$ matrices, **poor cache reuse**):
 - $I = \frac{2 \cdot N^3}{8 \cdot (2 \cdot N^3 + N^2)} = \frac{N^3}{8 \cdot N^3 + 4 \cdot N^2} \approx \frac{1}{8}$ FLOP/byte for large N
 - For large N , $P \leq \min\left(64, \frac{1}{8} \cdot 32\right) = 4$ GFLOPS
 - The textbook DGEMM is memory-bound in the worst case
- Tile-based DGEMM ($N \times N$ matrices, **ideal cache reuse**):
 - $I = \frac{n_{tiles} \cdot 2 \cdot T^3}{n_{tiles} \cdot 8 \cdot (3 \cdot T^2)} = \frac{T}{12}$ FLOP/byte
 - $P \leq \min\left(64, \frac{T}{12} \cdot 32\right) = 64$ GFLOPS with $T \geq 24$
 - The tile-based matrix-multiplication is compute-bound approaching peak with proper tile size

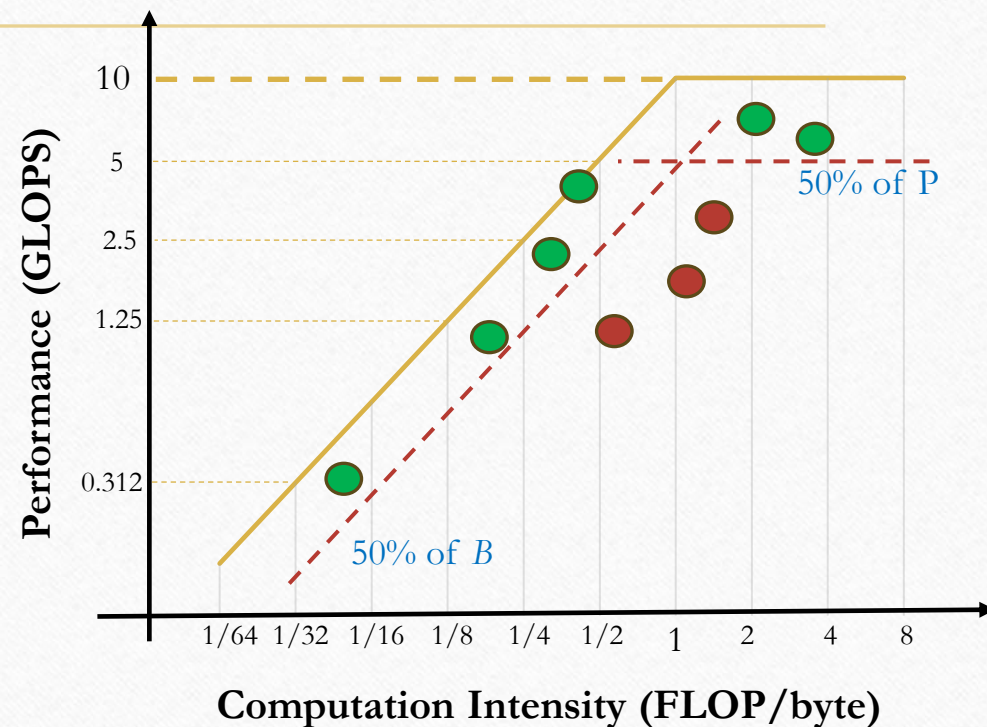
```
// Textbook DGEMM
for (int i=0; i<N; ++i)
  for (int j=0; j<N; ++j) {
    double s = 0.0;
    for (int k=0; k<N; ++k)
      s += A[i*N+k] * B[k*N+j]; // 2 FLOP
    C[i*N + j] = s;
  }
```

```
//Tile-based DGEMM
for (int i = 0; i < n; i += T)
  for (int k = 0; k < l; k += T)
    for (int j = 0; j < m; j += T) {
      for (int ii = i; ii < std::min(i + T,n); ++ii)
        for (int kk = k; kk < std::min(k + T,l); ++kk) {
          double a = A[ii * l + kk];
          for (int jj = j; jj < std::min(j + T,m); ++jj)
            C[ii * m + jj] += a * B[kk * m + jj];
        }
    }
}
```

Key takeaway: By reusing data in cache, I increases as $O(T)$, with an improvement of about $\frac{2}{3} \cdot T$ compared to textbook DGEMM

Multiple kernels in the applications

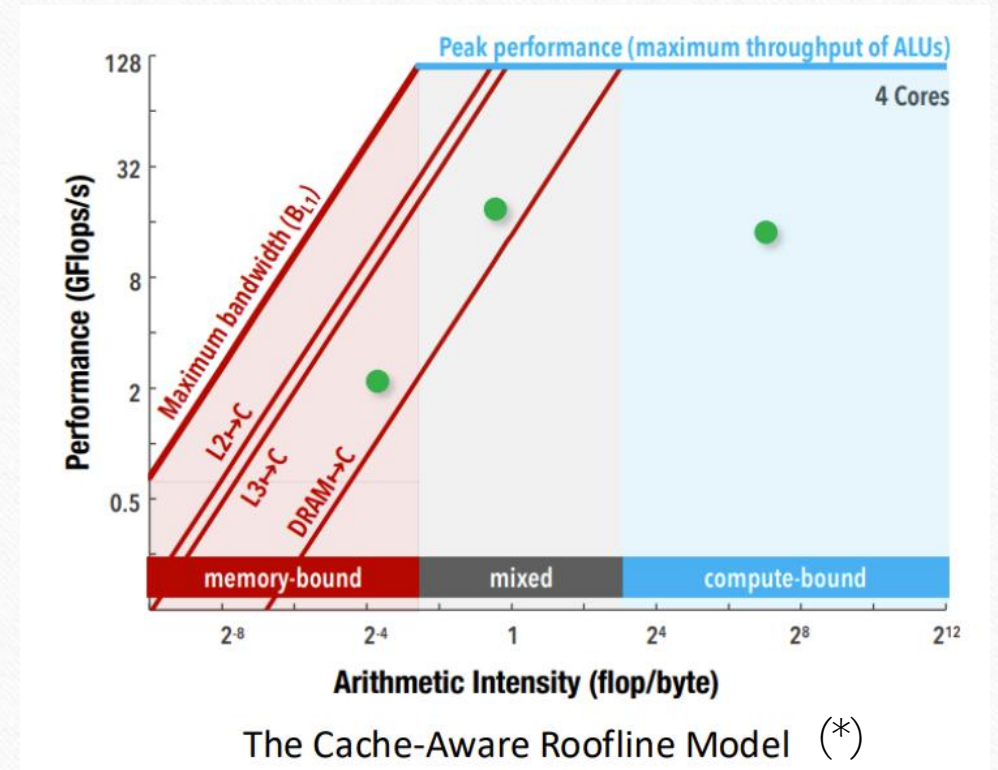
- Multiple points for each application kernel obtained by instrumentation of code
 - Some kernels may have low performance but they make good use of available bandwidth
 - Other kernels may have high performance but can make poor use of machine bandwidth
- It tells us that we have to prioritize the kernels whose points are in the poor-performance region (●)
- For those in the bandwidth bound region, we could try to reduce the data traffic (e.g., by changing algorithm)



- 50% of P: kernel achieves half of compute peak
- 50% of B: kernel achieves half of memory bandwidth

Cache-Aware Roofline Model

- Multiple levels of memory on modern CMPs
 - L1, L2, L3 caches, HBM, DDR, NVRAM
- Different bandwidths for each memory level, a kernel may be limited by different levels depending on I
- Each diagonal line represents a bandwidth limitation (slope) at a different memory level, and the flat line represents the peak compute performance
- Although a loop nest may have multiple AI's and multiple bounds for each memory level performance is bound by the minimum



(*) https://indico.cern.ch/event/1329693/contributions/5597312/attachments/2926483/5137468/CARM_Tool_CERN_Forum_11_9_24.pdf

Roofline model comments

- Based on two key concepts: **work** expressed in FLOPS (or any other meaningful metric), and **traffic**, which is the data required to perform the work (**B** must be measured through some benchmarks, e.g., the stream benchmark)
- Strong assumptions (**limitations**):
 - Data transfer and work perfectly overlap. **Reality**: memory stalls can block computation.
 - Steady-state execution, meaning there are no startup/shutdown effects: **Reality**: for small kernel the initial/fianal phases cost
 - Only bandwidth counts, the latency is completely ignored. **Reality**: random access patterns suffer from latency
 - Perfect cache behavior: **Reality**: conflict misses, TLB misses, coherence overhead, ...
- The Roofline model helps understand application performance relative to the machine capabilities and prioritizes optimizations
- General strategies suggested by the roofline analysis:
 - Maximize in-core performance (**through vectorization, loop unrolling, use of special instructions e.g., FMA**)
 - Maximize memory bandwidth (e.g., **NUMA-aware allocation, HW and SW prefetching, non-temporal stores**)
 - Minimize data movement, thus increasing I (**cache tiling, loop fusion, SoA vs AoS**)

General comments

- **Complex Environments:** HW details can be difficult or impossible to capture in purely theoretical models. The interplay between HW, compilers, OSs, and different workloads can lead to hard-to-predict performance figures
- **Theoretical Models as Guides:** Theoretical models (like Roofline model or Amdahl's/Gustafson's laws, which we will see later in the course) are still valuable for guiding initial designs and estimating upper performance bounds
 - **Start simple:** priority correctness, avoid premature optimization, avoid overengineering.
 - **Measure:** profile to find bottleneck (PAPI, perf, Vtune), measure cache miss rates and bandwidth. Then optimize
 - **Optimize:** focus on bottlenecks identified by profiling; one optimization at a time; measure the impact of each change
 - **Iterate:** re-test/profile after each optimization; understand when to stop (diminishing returns)
- “**Premature optimization is the root of all evil**” – Donald Knuth
- “**In theory, theory and practice are the same. In practice, they are not**” – Source unknown

Suggested Readings

- Chapter 3 of the Parallel Programming Concept and Practice book
- Cache-coherence protocols: «A Primer on Memory Consistency and Cache Coherence» Daniel J. Sorin, Mark D. Hill, and David A. Wood
- Roofline model: “Roofline: an insightful visual performance model for multicore architectures” by S. Williams, A. Waterman, D. Patterson, Communications of the ACM, Volume 52, Issue 4. Free access: <https://dl.acm.org/doi/pdf/10.1145/1498765.1498785>
- «Cache optimization techniques: What every programmer should know about memory» by Ulrich Drepper
- «Performance Analysis and Tuning on Modern CPUs» by Denis Bakhvalov
<https://github.com/dendibakh/perf-book>